

НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ОДЕСЬКА ПОЛІТЕХНІКА»

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

ІНСТИТУТ КОМП'ЮТЕРНИХ СИСТЕМ

КАФЕДРА ІНФОРМАЦІЙНИХ СИСТЕМ

Протокол лабораторної роботи №1

«розгортання середовища та порівняльний аналіз мовних моделей»

з дисципліни: «Обчислювальні методи»

Виконала: студентка групи AI-245

Ревуцька О.П.

Перевірів: ас. Баранов О.М.

Мета роботи

Запустити мовну модель локально та навчитися керувати нею з коду; ознайомитися з принципом взаємодії з локальними та хмарними мовними моделями через спільний інтерфейс; навчитися вимірювати продуктивність інференсу та аналізувати складові затримки; експериментально дослідити особливості роботи мовних моделей.

Хід роботи

Крок 1. Встановлення Ollama та завантаження моделей

Для локального запуску мовних моделей було використано програму Ollama. Вона працює як локальний сервіс та дозволяє завантажувати мовні моделі, запускати їх на комп'ютері та надсилати до них запити через API.

Після встановлення Ollama було виконано перевірку її роботи:

```
ollama --version
```

Далі було завантажено дві моделі:

```
ollama pull qwen3:4b
```

```
ollama pull llama3.2:3b
```

Для перевірки наявності моделей виконано:

```
ollama list
```

У результаті було встановлено:

- qwen3:4b — модель розміром приблизно 2,5 GB;
- llama3.2:3b — модель розміром приблизно 2,0 GB.

Рисунок 1 – Перевірка встановлених моделей за допомогою команди `ollama list`.

```
Microsoft Windows [Version 10.0.26200.9550]
(c) Microsoft Corporation. All rights reserved.

C:\Users\oleks>ollama pull qwen3:4b
pulling manifest
pulling 3e4cb1417446: 100% ▬ 2.5 GB
verifying sha256 digest
writing manifest
success

C:\Users\oleks>ollama pull llama3.2:3b
pulling manifest
pulling dde5aa3fc5ff: 100% ▬ 2.0 GB
pulling fcc5a6bec9da: 100% ▬ 7.7 KB
verifying sha256 digest
writing manifest
success

C:\Users\oleks>ollama list
NAME        ID                SIZE    MODIFIED
llama3.2:3b a80c4f17acd5     2.0 GB  12 seconds ago
qwen3:4b    359d7d94bcd5     2.5 GB  About a minute ago
```

Крок 2. Перевірка локального API Ollama

Ollama запускає локальний веб-сервер, доступний за адресою `http://localhost:11434`. Для перевірки можливості безпосереднього звернення до моделі через API було використано команду `curl`.

Приклад запиту:

```
curl http://localhost:11434/api/generate -d
'{"model":"qwen3:4b","prompt":"Столиця України?","stream":false}'
```

У результаті Ollama повертає відповідь у форматі JSON.

Основні поля отриманої відповіді:

- `response` — кінцева текстова відповідь моделі;
- `thinking` — внутрішні міркування моделі;
- `done` — ознака завершення генерації;
- `done_reason` — причина завершення;
- `load_duration` — час завантаження моделі в пам'ять;
- `prompt_eval_duration` — час обробки запиту;
- `eval_duration` — час генерації відповіді;
- `total_duration` — загальний час виконання запиту;
- `prompt_eval_count` — кількість токенів вхідного запиту;
- `eval_count` — кількість згенерованих токенів разом із внутрішніми міркуваннями.

Таким чином було перевірено, що локальна модель доступна через HTTP API і може використовуватися не тільки безпосередньо через Ollama, а й з інших програм.

Завдання 1. Ціна роздумів: міркувальна vs інструктивна модель

Мета завдання

Порівняти поведінку міркувальної моделі qwen3:4b та інструктивної моделі llama3.2:3b, визначити кількість згенерованих токенів, швидкість генерації, загальний час відповіді та дослідити стабільність моделей під час виконання задачі на міркування.

У методичних вказівках llama3.2:3b розглядається як інструктивна модель, яка безпосередньо формує відповідь, а qwen3:4b — як міркувальна модель, що перед кінцевою відповіддю створює внутрішню чернетку в полі thinking.

LP1_ЗАВДАННЯ

При порівнянні моделей необхідно розрізняти два показники:

токенів за секунду — характеризує швидкість генерації моделі на використаному обладнанні;

загальний час відповіді — залежить не тільки від швидкості, а й від обсягу роботи, яку виконує модель.

1. Просте фактичне питання

Спочатку було перевірено поведінку моделей на простому фактичному питанні.

Для обох моделей використано запит:

«Столиця України? Відповідай одним словом.»

Для моделі qwen3:4b виконано:

```
curl http://localhost:11434/api/generate -d  
{"model":"qwen3:4b","prompt":"Столиця України? Відповідай одним  
словом.","stream":false}
```

Для llama3.2:3b:

```
curl http://localhost:11434/api/generate -d
```

```
"{"model":"llama3.2:3b","prompt":"Столиця України? Відповідай одним словом.","stream":false}"
```

Такі запити передбачені першим кроком Завдання 1. ЛР1_ЗАВДАННЯ

Обидві моделі дали правильну відповідь — «Київ».

Однак поведінка моделей відрізнялася. qwen3:4b перед кінцевою відповіддю створила внутрішні міркування, тоді як llama3.2:3b сформулила коротку відповідь без такого великого обсягу додаткової генерації.

Рисунок 2 – Результати виконання фактичного запиту моделями qwen3:4b та llama3.2:3b.

```
C:\Users\oleks>curl http://localhost:11434/api/generate -d '{"model":"qwen3:4b","prompt":"Столиця України? Відповідай одним словом.","stream":false}'
{"model":"qwen3:4b","created_at":"2026-09-26T19:17:57.3148586Z","response":"Київ","thinking":"Хорошо, мне нужно определить столицу Украины и ответить одним словом. Я помню, что столица Украины – это Киев. Проверю, чтобы не ошибиться. В Украине столица называется Киев, и это правильно. Другие города, например, Харьков или Одесса, не являются столицей. Значит, правильный ответ – Киев.\n","done":true,"done_reason":"stop","context":[351644,872,198,73498,7819,1882,10373,4235,127072,144831,22621,30,22933,26289,6949,132502,26289,6949,18530,138502,91107,12228,13,151645,198,151644,77091,198,151567,198,129942,125234,12150,1456,11,129075,129100,45239,91482,17787,130297,1802,134230,136727,7587,92647,17787,138502,91107,12228,13,128557,75301,127494,11,47389,130297,1802,86894,136727,1950,67879,127465,144031,5474,11,22419,68959,11016,11,81150,18658,88663,128708,15,22933,137829,1504,130297,1802,86894,127550,139916,138868,11,7587,67879,135544,13,38803,64389,49206,131644,11,90005,2184,80348,11,125560,134187,89777,45077,132593,93420,1478,11,18658,134958,130297,1802,10373,21832,13,52577,30181,9358,11,72126,66502,33513,92647,1959,138868,624,151668,271,26338,1802,144873,5474],"total_duration":26621758400,"load_duration":14317650300,"prompt_eval_count":30,"prompt_eval_cached_count":0,"prompt_eval_duration":1419097000,"eval_count":101,"eval_duration":10057381000}
C:\Users\oleks>curl http://localhost:11434/api/generate -d '{"model":"llama3.2:3b","prompt":"Столиця України? Відповідай одним словом.","stream":false}'
{"model":"llama3.2:3b","created_at":"2026-09-26T19:18:47.5045889Z","response":"Київ","done":true,"done_reason":"stop","context":[128006,9125,128007,271,38766,1303,13025,2696,25,6798,220,2366,18,271,128009,128006,882,128007,271,74598,181745,181820,181737,30,186963,182105,19039,111657,92207,125807,13,128009,128006,28191,128007,271,27435,1840,185598,13],"total_duration":18440242600,"load_duration":17094036700,"prompt_eval_count":37,"prompt_eval_cached_count":0,"prompt_eval_duration":1028622000,"eval_count":5,"eval_duration":315716000}
C:\Users\oleks>
```

2.

Розрахунок продуктивності

Тривалості в JSON-відповіді Ollama наведено в наносекундах. Для переведення загального часу в секунди використано формулу:

Час, с = total_duration / 10⁹

Швидкість генерації визначалася за формулою:

Токенів/с = eval_count / (eval_duration / 10⁹)

Саме ці формули наведено в методичних вказівках. ЛР1_ЗАВДАННЯ

За отриманими результатами було сформовано таблицю.

Таблиця 1 – Фактичне питання

Показник qwen3:4b llama3.2:3b

response Київ Київ

Показник qwen3:4b llama3.2:3b

eval_count 101 5

Час, с 26,62 18,45

Токенів/с 9,30 15,84

Обидві моделі правильно відповіли на поставлене питання. Водночас qwen3:4b згенерувала 101 токен, а llama3.2:3b — лише 5 токенів.

Таким чином, для отримання практично однакового кінцевого результату міркувальна модель виконала значно більший обсяг генерації.

3. Задача на міркування

Наступним етапом було перевірено поведінку моделей на задачі, яка потребує обчислення.

Використано запит:

«Ручка і зошит разом коштують 110 грн. Ручка на 100 грн дешевша за зошит. Скільки коштує ручка?»

Правильна відповідь — 5 грн.

Дійсно, якщо позначити ціну ручки через x , то ціна зошита дорівнює $x + 100$.

Тоді:

$$x + (x + 100) = 110$$

$$2x + 100 = 110$$

$$2x = 10$$

$$x = 5 \text{ грн.}$$

За умовами методичних вказівок кожна модель повинна виконати цю задачу 5 разів. ЛР1_ЗАВДАННЯ

Для qwen3:4b виконувалася команда:

```
curl http://localhost:11434/api/generate -d
```

```
"{\"model\":\"qwen3:4b\", \"prompt\":\"Ручка і зошит разом коштують 110 грн.
```

Ручка на 100 грн дешевша за зошит. Скільки коштує ручка?\", \"stream\":false}"

Для llama3.2:3b:

```
curl http://localhost:11434/api/generate -d
```

```
"{"model":"llama3.2:3b","prompt":"Ручка і зошит разом коштують 110 грн. Ручка на 100 грн дешевша за зошит. Скільки коштує ручка?\", \"stream\":false}"
```

4. Результати qwen3:4b

Для міркувальної моделі було проведено п'ять запусків .

Таблиця 2.1 – Результати запусків qwen3:4b

№ запуску eval_count Загальний час, с

1	1226	175,53
2	1077	139,67
3	1194	132,64
4	1374	165,05
5	1382	163,01
Середнє	1250,6	155,18

Середня кількість згенерованих токенів:

$(1226 + 1077 + 1194 + 1374 + 1382) / 5 = 1250,6$ токена

Середній час:

$(175,53 + 139,67 + 132,64 + 165,05 + 163,01) / 5 = 155,18$ с

Таким чином, один запуск qwen3:4b займав у середньому приблизно 155 секунд, або близько 2 хв 35 с.

На отриманих результатах добре видно великий обсяг поля thinking. Модель генерувала значну кількість проміжного тексту перед остаточною відповіддю.

Рисунки 3–7 – Результати п'яти запусків qwen3:4b.

```
C:\WINDOWS\system32\cmd.exe
89, 55899, 13685, 134174, 11, 68371, 53470, 130960, 18108, 28371, 13132, 13, 4710, 98574, 5474, 125033, 94269, 12228, 11, 133849, 126083, 13048, 21228, 19645, 30181, 17787, 0
6099, 42965, 13, 38803, 28156, 60513, 16104, 11, 141385, 34492, 133599, 18108, 28371, 16748, 68823, 44558, 140911, 26991, 5259, 11, 20396, 133599, 10885, 1456, 12150, 130141
, 1959, 13932, 140911, 26991, 5259, 13, 50318, 124846, 16786, 127757, 128537, 133771, 1802, 13932, 126460, 79165, 130876, 13932, 26988, 2108, 18492, 1447, 16, 13, 44558, 488,
13932, 284, 220, 16, 15, 320, 5474, 126819, 1504, 128729, 18362, 60208, 1792, 220, 16, 15, 15, 140912, 340, 17, 13, 13932, 284, 44558, 488, 220, 16, 15, 15, 320, 132633, 12150,
9358, 131487, 21259, 1504, 18108, 28371, 16748, 13073, 220, 16, 13, 50318, 37421, 45310, 39742, 39111, 15952, 11, 47389, 46253, 18108, 28371, 13132, 7796, 45967, 32642, 12476
3, 10885, 1456, 12150, 130141, 13073, 220, 16, 15, 15, 11, 58246, 10885, 1456, 12150, 9358, 130960, 13073, 220, 16, 15, 15, 129258, 11, 128743, 18108, 28371, 13132, 13, 131345, 7
9165, 13932, 284, 44558, 488, 220, 16, 15, 15, 13, 22419, 68959, 11916, 25, 46253, 18108, 28371, 13132, 130960, 220, 17, 15, 11, 58246, 10885, 1456, 12150, 9358, 220, 16, 17, 15, 1
1, 134564, 38379, 131692, 220, 16, 15, 15, 13, 34789, 8178, 11, 134564, 13932, 26988, 2108, 17175, 136648, 61631, 13932, 284, 44558, 488, 220, 16, 15, 15, 30, 34789, 8178, 11, 397
42, 21259, 126218, 13, 71144, 18108, 28371, 13132, 7796, 45967, 32642, 124763, 10885, 1456, 12150, 130141, 13073, 220, 16, 15, 15, 11, 58246, 10885, 1456, 12150, 9358, 130960,
129258, 13073, 220, 16, 15, 15, 13, 131345, 79165, 11, 90005, 2184, 80348, 11, 46253, 18108, 28371, 13132, 130960, 220, 18, 15, 11, 58246, 10885, 1456, 12150, 9358, 220, 16, 18, 1
5, 13, 50318, 124846, 38379, 131692, 220, 16, 15, 15, 13, 52577, 30181, 9358, 11, 13932, 26988, 2108, 17175, 13932, 284, 44558, 488, 220, 16, 15, 15, 63262, 13685, 382, 33995, 660
12, 127624, 39742, 67274, 16104, 138616, 43767, 13932, 26988, 2108, 17175, 5805, 76801, 43767, 13, 129977, 138616, 22107, 13932, 26988, 2108, 18492, 13932, 284, 44558, 488, 2
20, 16, 15, 15, 13, 130841, 67274, 16104, 5805, 76801, 43767, 25, 44558, 488, 320, 10474, 488, 220, 16, 15, 15, 8, 284, 220, 16, 15, 13, 131345, 79165, 220, 17, 10474, 488, 220, 1
6, 15, 15, 284, 220, 16, 15, 13, 50318, 124846, 220, 17, 10474, 284, 220, 16, 15, 15, 481, 220, 16, 15, 15, 284, 220, 16, 15, 11, 138721, 44558, 284, 220, 20, 13, 131345, 79165, 181
08, 28371, 13132, 130960, 220, 20, 140912, 11, 50318, 124846, 10885, 1456, 12150, 9358, 130960, 220, 20, 488, 220, 16, 15, 15, 284, 220, 16, 15, 15, 140912, 11, 66988, 45310, 129162, 2
, 24634, 80
663, 20562, 30, 140085, 11, 5805, 127757, 46173, 5409, 133066, 1456, 330, 131586, 13132, 13073, 220, 16, 15, 15, 140912, 7796, 45967, 32642, 128805, 43846, 10885, 1456, 12150,
9358, 3263, 131345, 79165, 18108, 28371, 13132, 7796, 45967, 32642, 124763, 10885, 1456, 12150, 130141, 13073, 220, 16, 15, 15, 11, 58246, 79165, 10885, 1456, 12150, 9358, 131
487, 21259, 1504, 18108, 28371, 16748, 13073, 220, 16, 15, 15, 13, 50318, 124846, 91810, 11, 13932, 284, 44558, 488, 220, 16, 15, 15, 13, 50318, 124846, 134746, 63262, 47050, 382
, 20195, 1456, 135927, 5805, 133066, 133771, 126282, 141615, 38003, 129348, 61631, 7587, 42965, 11, 128330, 131337, 17686, 132114, 1504, 11, 47389, 92647, 220, 20, 140912, 13
, 22419, 68959, 11916, 128863, 38379, 382, 55091, 32202, 18108, 28371, 13132, 220, 20, 11, 10885, 1456, 12150, 9358, 220, 16, 15, 20, 13, 274999, 39111, 68363, 220, 16, 16, 15, 13,
127088, 131692, 220, 16, 15, 20, 481, 220, 20, 284, 220, 16, 15, 15, 13, 129117, 135544, 13, 4710, 16206, 126284, 1504, 37660, 43993, 5409, 30801, 96234, 13, 128543, 66988, 61631
, 11, 45310, 6709, 56824, 26988, 127764, 141638, 4235, 3038, 127757, 49206, 30, 140085, 11, 46253, 37660, 128656, 90005, 13502, 7336, 1456, 11, 47389, 10885, 1456, 12150, 9358
, 7796, 45967, 32642, 124763, 18108, 28371, 16748, 13073, 220, 16, 15, 15, 11, 58246, 92647, 128897, 37660, 135361, 13, 128543, 5805, 127757, 46173, 5409, 133066, 1456, 25, 330
, 131586, 13132, 13073, 220, 16, 15, 15, 140912, 7796, 45967, 32642, 128805, 43846, 10885, 1456, 12150, 9358, 3263, 131345, 79165, 18108, 28371, 13132, 7796, 45967, 32642, 124
763, 10885, 1456, 12150, 130141, 13073, 220, 16, 15, 15, 13, 131345, 79165, 10885, 1456, 12150, 9358, 131487, 21259, 1504, 13, 4710, 24110, 1478, 11, 43993, 135544, 13, 50318, 1
24846, 92647, 220, 20, 140912, 382, 20195, 1456, 5805, 141059, 133771, 126282, 129348, 61631, 39742, 128399, 129713, 13, 140085, 11, 46253, 37660, 143433, 68363, 129760, 220
, 16, 16, 15, 11, 20396, 38379, 131692, 220, 16, 15, 15, 11, 58246, 143433, 68363, 136230, 61631, 129258, 38379, 134793, 13, 22933, 141536, 129402, 143433, 68363, 220, 16, 16, 15
, 11, 38379, 131692, 220, 16, 15, 15, 11, 58246, 79165, 18108, 28371, 13132, 137201, 10885, 1456, 12150, 130141, 13073, 220, 16, 15, 15, 11, 138721, 11, 18108, 28371, 13132, 6882
3, 320, 16, 15, 481, 220, 16, 15, 15, 5620, 17, 284, 220, 20, 11, 10885, 1456, 12150, 9358, 220, 16, 15, 20, 13, 129279, 11, 67879, 137255, 33513, 55629, 136080, 133360, 133771, 1
382, 24110, 1478, 11, 43993, 63262, 13685, 13, 38803, 130681, 11916, 11, 92647, 220, 20, 140912, 624, 151668, 271, 24110, 13695, 18108, 48755, 6, 62984, 7336, 39151, 133771, 26
289, 5805, 37722, 9662, 34011, 1478, 144073, 126242, 13073, 126399, 2108, 26289, 97499, 26289, 1447, 16, 13, 48420, 28371, 13132, 83854, 10885, 1456, 12150, 9358, 38379, 1222
8, 87415, 12150, 1792, 13888, 28793, 220, 16, 15, 15, 140912, 624, 17, 13, 48420, 28371, 13132, 13073, 220, 16, 15, 15, 140912, 7796, 45967, 32642, 128805, 43846, 10885, 1456, 12
150, 9358, 382, 16854, 19645, 30181, 16104, 1456, 38133, 26289, 125467, 18108, 28371, 16748, 45310, 4793, 400, 856, 400, 320, 124672, 2108, 701, 20396, 38133, 26289, 125467, 1
0885, 1456, 12150, 130141, 45310, 4793, 400, 379, 400, 320, 124672, 2108, 508, 50318, 9516, 16289, 128092, 144873, 126242, 91999, 3780, 18108, 26289, 5474, 39151, 126811, 144
7, 14085, 198, 59, 7265, 90, 23910, 532, 87, 488, 379, 284, 220, 16, 16, 15, 90155, 88, 284, 856, 488, 220, 16, 15, 15, 108, 50, 408, 90, 23910, 532, 14085, 271, 16854, 26280, 6949, 67
274, 13695, 144073, 126242, 400, 379, 284, 856, 488, 220, 16, 15, 15, 400, 13932, 27016, 58909, 18108, 26289, 5474, 39151, 2108, 39151, 1447, 14085, 198, 87, 488, 320, 87, 488, 22
8, 16, 15, 15, 8, 284, 220, 16, 15, 15, 198, 14085, 271, 14085, 198, 17, 87, 488, 220, 16, 15, 15, 284, 220, 16, 15, 198, 14085, 271, 14085, 198, 17, 87, 284, 220, 16, 15, 198, 14085,
271, 14085, 198, 87, 284, 220, 20, 198, 14085, 271, 59527, 58038, 11, 38133, 26289, 14191, 18108, 28371, 16748, 148661, 17787, 400, 1124, 79075, 90, 20, 92, 400, 140912, 13, 1, "to
tal_duration":175529577300, "load_duration":21701700, "prompt_eval_count":63, "prompt_eval_cached_count":3, "prompt_eval_duration":248431000, "eval_coun
t":1226, "eval_duration":172993620000}
C:\Users\oleks>
```

```
13, 379, 284, 856, 488, 220, 16, 15, 15, 320, 8005, 22787, 127266, 35252, 18108, 28371, 13132, 7796, 45967, 32642, 124763, 10885, 1456, 220, 16, 15, 15, 140912, 11, 58246, 10885,
1456, 12150, 9358, 13073, 220, 16, 15, 15, 140912, 131487, 21259, 1504, 18108, 28371, 16748, 692, 20195, 1456, 10813, 37421, 45310, 52189, 39111, 15952, 20562, 25, 46253, 1810
8, 28371, 13132, 7796, 45967, 32642, 124763, 10885, 1456, 12150, 130141, 13073, 220, 16, 15, 15, 140912, 11, 58246, 67879, 8215, 129974, 26909, 11, 47389, 10885, 1456, 12150, 9
358, 130960, 13073, 220, 16, 15, 15, 13, 140912, 131487, 21259, 1504, 18108, 28371, 16748, 11, 131345, 79165, 11, 46253, 18108, 28371, 13132, 130960, 856, 11, 58246, 10885, 1456, 1
2150, 9358, 130960, 856, 488, 220, 16, 15, 15, 13, 50318, 124846, 39742, 67274, 16104, 67879, 5805, 76801, 43767, 13932, 26988, 2108, 17175, 382, 30174, 1792, 15860, 11, 856, 48
8, 320, 87, 488, 220, 16, 15, 15, 8, 284, 220, 16, 16, 15, 13, 4710, 16854, 68959, 11916, 25, 51132, 67171, 220, 17, 87, 488, 220, 16, 15, 15, 284, 220, 16, 16, 15, 13, 50318, 124846, 22
8, 17, 87, 284, 220, 16, 15, 15, 481, 220, 16, 15, 15, 284, 220, 16, 15, 11, 138721, 856, 284, 220, 16, 15, 13, 131345, 79165, 18108, 28371, 13132, 130960, 220, 20, 140912, 13, 50318, 124
846, 10885, 1456, 12150, 9358, 130960, 220, 20, 488, 220, 16, 15, 15, 284, 220, 16, 15, 20, 140912, 13, 22419, 68959, 16104, 25, 220, 20, 488, 220, 16, 15, 20, 284, 220, 16, 15, 15, 13
, 129279, 11, 131896, 5409, 30801, 96234, 382, 20195, 1456, 39742, 21259, 126218, 11, 5805, 127757, 46173, 5409, 133066, 1456, 330, 131586, 13132, 13073, 220, 16, 15, 15, 14091
2, 7796, 45967, 32642, 128805, 43846, 10885, 1456, 12150, 9358, 3263, 131345, 79165, 11, 18108, 28371, 13132, 7796, 45967, 32642, 124763, 10885, 1456, 12150, 130141, 13073, 2
20, 16, 15, 15, 140912, 13, 131345, 79165, 11, 10885, 1456, 12150, 9358, 131487, 21259, 1504, 18108, 28371, 16748, 13073, 220, 16, 15, 15, 140912, 13, 52577, 30181, 9358, 11, 139
52, 26988, 2108, 17175, 379, 284, 856, 488, 220, 16, 15, 15, 63262, 13685, 13, 50318, 124846, 131896, 135544, 382, 20195, 1456, 134564, 92647, 220, 20, 140912, 11, 30, 22419, 68959,
11916, 134413, 38379, 13, 71144, 18108, 28371, 13132, 220, 20, 11, 10885, 1456, 12150, 9358, 220, 16, 15, 20, 11, 143433, 68363, 220, 16, 16, 15, 13, 129279, 11, 7507, 38379, 1316
92, 220, 16, 15, 15, 13, 22933, 2247, 44022, 135544, 382, 20195, 1456, 66988, 61631, 11, 45310, 47389, 11, 24634, 90005, 37421, 15952, 30, 140085, 11, 5805, 127757, 46173, 310, 1
31586, 13132, 13073, 220, 16, 15, 15, 140912, 7796, 45967, 32642, 128805, 43846, 10885, 1456, 12150, 9358, 1, 1959, 58246, 79165, 11, 18108, 28371, 13132, 130960, 13073, 220, 1
6, 15, 15, 140912, 137201, 11, 128743, 10885, 1456, 12150, 9358, 13, 131345, 79165, 11, 10885, 1456, 12150, 9358, 131487, 21259, 1504, 18108, 28371, 16748, 13073, 220, 16, 15, 1
5, 13, 129279, 11, 58246, 79165, 67879, 51670, 38379, 379, 284, 856, 488, 220, 16, 15, 15, 382, 16206, 2247, 44022, 63262, 13685, 13, 50318, 124846, 92647, 220, 20, 140912, 382, 2
0195, 1456, 45310, 75301, 127494, 11, 5805, 141059, 133771, 126202, 39742, 13764, 38800, 137796, 66988, 61631, 6709, 11916, 126382, 11, 128330, 131337, 131896, 143846, 1290
20, 125033, 13, 38803, 28156, 60513, 16104, 11, 46253, 37660, 128656, 13073, 139243, 11, 58246, 79165, 10885, 1456, 12150, 9358, 7796, 45967, 32642, 124763, 11, 128330, 5805,
127757, 46173, 135151, 5409, 133066, 1456, 11, 47389, 18108, 28371, 13132, 7796, 45967, 32642, 124763, 10885, 1456, 12150, 130141, 13, 131345, 79165, 11, 10885, 1456, 12150,
9358, 131487, 21259, 1504, 382, 20195, 1456, 127757, 37660, 128656, 330, 132633, 12150, 9358, 13073, 220, 16, 15, 15, 140912, 7796, 45967, 32642, 58909, 18108, 28371, 167
48, 497, 58246, 134564, 37660, 128656, 856, 284, 379, 488, 220, 16, 15, 15, 11, 128330, 131337, 13073, 139243, 382, 24110, 1478, 11, 131896, 135544, 13, 52577, 30181, 9358, 11, 1
8108, 28371, 13132, 130960, 220, 20, 140912, 382, 334, 59527, 46974, 1819, 33504, 28371, 13132, 87415, 12150, 88248, 144073, 1124, 79075, 90, 20, 92, 140912, 624, 151668, 271,
24110, 13695, 18108, 48755, 6, 62984, 7336, 39151, 133771, 26289, 5805, 37722, 9662, 34011, 1478, 144073, 126242, 13073, 126399, 2108, 26289, 97499, 26289, 1447, 16, 13, 4842
0, 28371, 13132, 83854, 10885, 1456, 12150, 9358, 38379, 12228, 87415, 12150, 1792, 13048, 28793, 220, 16, 16, 15, 140912, 624, 17, 13, 48420, 28371, 13132, 13073, 220, 16, 15, 1
5, 140912, 7796, 45967, 32642, 128805, 43846, 10885, 1456, 12150, 9358, 382, 16854, 19645, 30181, 16104, 1456, 510, 12, 400, 856, 400, 1959, 38133, 26289, 14191, 18108, 28371,
16748, 320, 5474, 140912, 1326, 12, 400, 379, 400, 1959, 38133, 26289, 14191, 10885, 1456, 12150, 130141, 320, 5474, 140912, 3593, 35451, 13932, 6442, 6715, 1802, 133771, 2628
9, 128602, 144073, 126242, 130876, 18108, 26289, 5474, 39151, 2108, 39151, 510, 16, 13, 400, 856, 488, 379, 284, 220, 16, 16, 15, 24437, 17, 13, 400, 379, 284, 856, 488, 220, 16, 15
, 15, 84500, 16854, 26289, 6949, 67274, 16104, 1456, 80742, 1504, 18108, 26289, 5474, 39151, 2108, 39151, 5805, 27016, 58909, 510, 14085, 198, 87, 488, 320, 87, 488, 220, 16, 15,
15, 8, 284, 220, 16, 15, 198, 14085, 198, 14085, 198, 17, 87, 488, 220, 16, 15, 15, 284, 220, 16, 15, 198, 14085, 198, 14085, 198, 17, 87, 284, 220, 16, 15, 198, 14085, 198, 14085
, 198, 87, 284, 220, 20, 198, 14085, 271, 33995, 158
```

```
C:\WINDOWS\system32\cmd.exe
8066,1456,11,8839,231,1456,18108,28371,13132,13073,220,16,15,15,140912,7796,45967,32642,128805,43846,10885,1456,12150,9358,13,34789,55899,13685,1431
89,18530,22496,38133,26289,125467,18108,28371,16748,382,98574,58909,11,8839,231,1456,11310,26289,15398,36305,126910,1802,13073,13932,5474,13393,3780
1365,38133,1504,126515,11,8839,231,1456,11310,1802,128692,144073,126242,73154,26289,142233,26289,6949,12228,26289,138486,127228,1802,25,1813,33,26289
14191,18108,28371,16748,83854,38133,26289,14191,10885,1456,12150,130141,13,34789,97185,18530,38133,26289,14191,18108,28371,16748,5470,1504,856,140
912,11,20396,38133,26289,14191,10885,1456,12150,130141,1305,379,140912,13,58018,9516,26289,11,10885,15390,26289,6949,13685,10885,13932,6442,127015,1
1916,11,128692,144073,126242,130876,18108,26289,5474,39151,2108,39151,1447,16,13,856,488,379,284,220,16,15,320,14144,1456,18108,28371,13132,83854
10885,1456,12150,9358,38379,12228,87415,12150,1792,130848,28793,220,16,15,15,140912,340,17,33,379,284,856,488,220,16,15,320,126394,18108,28371,13132
132,13073,220,16,15,15,140912,7796,45967,32642,128805,43846,10885,1456,12150,9358,11,108813,13764,24634,10885,1456,12150,9358,131487,21259,6028,4
838,13073,220,16,15,15,140912,692,20353,6028,32642,47162,13685,11,8839,231,1456,80742,1504,18108,26289,5474,39151,2108,39151,5805,51574,3780,144073,3
13073,126515,11,8839,231,1456,38133,26289,14191,10885,1456,12150,130141,5805,26289,6949,128928,13685,18108,28371,16748,13073,220,16,15,15,140912,140
62,26289,28018,58909,13,58018,71852,11,45310,4793,14144,1456,11310,1802,5603,26289,6949,67274,16104,1456,379,10885,80742,21207,18108,26289,5474,3915
1,2108,39151,5805,27016,58909,11,58246,20264,2184,16104,1478,14073,126242,18108,26289,5474,39151,2108,39151,10885,52110,2108,26289,144073,119142,3
323,26289,6949,12228,1456,11016,382,24110,17753,18530,29780,139501,10090,1456,1447,87,488,320,87,488,220,16,15,8,284,220,16,15,271,33995,66024,7
7599,18108,19645,132347,144073,126242,7796,55899,16748,1447,87,488,856,488,220,16,15,15,284,220,16,15,271,17,87,488,220,16,15,284,220,16,15,16,16,1
5,271,33995,66024,77599,5805,26289,6949,2108,26289,6442,10090,1456,220,16,15,15,5805,26289,6949,21229,47162,5474,66645,127174,18943,1447,17,87,284,22
0,16,15,15,488,220,16,15,271,87,284,220,16,15,271,33995,66024,77599,7796,26289,18073,126242,21229,47162,5474,26289,127174,18943,1802,13073,220,1
7,1447,87,284,220,20,271,59527,58038,11,38133,26289,14191,18108,28371,16748,1365,220,20,140912,13,56755,87854,1456,12150,9358,87415,12150,88248,144
873,220,16,15,15,488,220,20,284,220,16,15,20,140912,13,135067,32642,26289,2184,16104,1456,25,220,20,488,220,16,15,20,284,220,16,15,11,8839,231,14
56,5805,26289,6949,132502,26289,39490,144073,11932,6442,6715,26289,13,1278,220,18108,28371,13132,13073,220,16,15,15,140912,7796,45967,32642,128805,4
3846,10885,1456,12150,9358,13685,220,16,15,20,481,220,20,284,220,16,15,15,11,38133,1504,49845,21259,5805,26289,2184,13685,382,36155,124703,17307,180
18658,14062,3780,38765,75301,63192,16748,38,34780,10077,36033,126647,11,17307,1802,133544,17108,19645,43513,13,12150,108832,1456,13932,6442,6715,3780
38,58742,133771,26289,5409,133066,1456,25,330,33580,28371,13132,13073,220,16,15,15,140912,7796,45967,32642,128805,43846,10885,1456,12150,9358,407,130
813,13764,24634,10885,1456,12150,9358,131487,21259,6028,43838,13073,220,16,15,15,140912,11,8839,231,1456,5805,26289,6949,132502,26289,39490,144073,1
26996,71852,18108,26289,5474,39151,2108,127494,379,284,856,488,220,16,15,15,13,125658,11,43993,133554,382,37991,21259,129618,26289,12141,75301,63192
47099,11916,14062,3780,38765,14062,11,45310,4793,14144,1456,14062,93036,6949,68675,75301,26289,54423,26289,14384,18108,28371,35252,7796,45967,32642,13262
3,43846,10885,1456,12150,9358,13073,220,16,15,15,140912,11,20396,124763,10885,1456,12150,9358,14062,126056,14062,1802,14062,26289,28018,58909,13,567
55,124763,5805,13932,6442,6715,26289,17307,26289,1792,47099,5409,133066,1456,11,8839,231,1456,18108,28371,13132,13073,220,16,15,15,140912,7796,45967
32642,128805,43846,10885,1456,12150,9358,11,20396,124763,10885,1456,11,3813
```

```
C:\WINDOWS\system32\cmd.exe
9,16,15,15,30,34789,8178,11,220,16,16,15,140912,11,49845,47189,856,488,179,284,220,16,16,15,382,16206,16673,43767,127757,49206,25,18108,28371,13132,
13073,220,16,15,15,140912,7796,45967,32642,124763,10885,1456,12150,130141,13,131345,79165,133509,18108,28371,16748,13073,220,16,15,15,137201,134072,
10885,1456,12150,130141,13,50318,124846,379,284,856,488,220,16,15,15,30,42796,11073,856,284,379,481,220,16,15,15,30,34789,55899,13685,39742,30111,17
998,382,55091,32202,18108,28371,13132,7796,45967,32642,124763,10885,1456,12150,130141,13073,220,16,15,15,140912,11,58246,10885,1456,12150,9358,13096
0,13073,220,16,15,15,140912,131487,21259,1504,18108,28371,16748,13,52577,30181,9358,11,379,284,856,488,220,16,15,15,13,129279,11,67879,135544,13,131
345,79165,133599,10885,1456,12150,130141,135933,14191,129827,1504,18108,28371,16748,5063,36806,2247,220,16,15,15,382,33995,66024,127624,13932,126460
,79165,134450,13932,26988,2108,61391,1447,16,8,856,488,379,284,220,16,16,15,271,17,8,379,284,856,488,220,16,15,15,271,20195,55899,13685,126091,17787
,135481,91999,3708,13,38803,17753,131646,39742,67274,16104,138616,43767,13932,26988,2108,17175,5805,76801,43767,382,143822,67274,66668,379,23064,139
32,26988,2108,18492,220,17,5805,13932,26988,2108,17175,220,16,1447,87,488,320,87,488,220,16,15,15,8,284,220,16,16,15,271,40582,124436,91406,30962,14
47,87,488,856,488,220,16,15,15,284,220,16,16,15,271,17,87,488,220,16,15,15,284,220,16,16,15,271,33995,66024,127624,20811,90253,30962,220,16,15,15,23
064,21229,1504,66645,127174,21032,1447,17,87,284,220,16,16,15,481,220,16,15,15,271,17,87,284,220,16,15,271,16854,13039,12228,7796,60542,6442,13073,2
20,17,1447,87,284,220,20,271,35451,30181,9358,11,18108,28371,13132,130960,220,20,140912,382,16854,68959,89775,13,71144,18108,28371,13132,130960,220,
20,140912,11,58246,10885,1456,12150,9358,130960,220,20,488,220,16,15,15,284,220,16,15,20,140912,13,50318,124846,143433,68363,220,20,488,220,16,15,20
,284,220,16,16,15,140912,13,129117,5409,30001,96234,13,52577,30181,9358,11,92647,220,20,140912,382,20195,1456,39742,21259,126218,11,5805,127757,4617
5,5409,133066,1456,330,131586,13132,13073,220,16,15,15,140912,7796,45967,32642,128805,43846,10885,1456,12150,9358,3263,131345,79165,18108,28371,1313
2,7796,45967,32642,124763,10885,1456,12150,130141,13073,220,16,15,15,140912,13,131345,79165,10885,1456,12150,9358,131487,21259,1504,18108,28371,1674
8,13073,220,16,15,15,13,50318,124846,91810,11,46253,18108,28371,13132,220,20,11,58246,10885,1456,12150,9358,220,16,15,20,11,143433,68363,220,16,16,1
5,13,129117,63262,13685,382,20195,1456,5805,6949,64389,45318,129162,12,24634,88663,20562,30,38803,17753,131646,128863,38379,68456,16104,382,55091,32
202,856,1959,18108,28371,13132,11,379,1959,10885,1456,12150,9358,382,87,488,379,284,220,16,16,15,271,88,284,856,488,220,16,15,15,271,143822,67274,66
668,25,856,488,856,488,220,16,15,15,284,220,16,16,15,11397,220,17,87,284,220,16,15,11397,856,20,20,13,129279,11,43993,135544,382,37991,128147,61633,
11,5805,133771,1504,45239,55757,7972,13132,30,22419,13039,71852,47389,135927,132842,133771,1802,5409,30379,131182,21032,5805,130157,125543,20562,179
2,89777,11,128330,131337,220,16,15,15,140912,38379,131692,11,20396,143433,68363,220,16,16,15,13,50318,124846,51332,67171,11,47389,10885,1456,12150,9
358,131487,21259,1504,13073,220,16,15,15,11,20396,18108,28371,13132,5805,220,20,13,50318,124846,143433,68363,220,16,15,20,488,20,20,16,16,15,13,1291
17,63262,13685,382,20195,1456,5805,124619,30827,67642,11,6709,17753,7599,47050,11,10885,1456,12150,9358,130960,220,16,15,20,140912,11,128330,5
805,127757,46173,133771,1802,49845,7587,79165,13,143711,11,67879,134800,43758,133771,1478,11,120162,40076,30343,62129,34582,4552,49845,11,81150,5133
2,15952,98720,129151,43767,64519,13,125658,47389,92647,220,20,140912,382,129942,125234,12150,1456,11,45318,7796,130601,11916,11,43993,135544,382,334
5,59527,46974,1019,33504,28371,13132,87415,12150,88248,144073,1124,79075,90,20,92,140912,624,151668,271,24110,13695,18108,48755,6,62984,7336,39151,13
3771,26289,5805,37722,9062,34011,1478,144073,126242,13073,126399,2108,26289,97499,26289,1447,16,13,48420,28371,13132,83854,10885,1456,12150,9358,383
79,12228,87415,12150,1792,33048,28793,220,16,16,15,140912,624,17,13,48420,28371,13132,13073,220,16,15,15,140912,7796,45967,32642,128805,43846,10885,
1456,12150,9358,382,16854,19645,30181,16104,1456,30133,26289,125467,18108,28371,16748,45310,4793,400,856,400,320,124672,2108,701,20396,30133,26289,1
25467,10885,1456,12150,130141,45310,4793,400,379,400,320,124672,2108,568,52577,13932,6442,6715,133771,26289,128602,144073,126242,91999,3780,18108,26
289,5474,39151,126811,1447,14085,198,59,7265,90,23910,532,87,488,379,284,220,16,16,15,15,90155,88,284,856,488,220,16,15,15,198,59,408,90,23910,532,14085
5,271,16854,26289,6949,67274,13695,144073,126242,400,379,284,856,488,220,16,15,15,400,13932,27016,58909,18108,26289,5474,39151,2108,39151,1447,14085
,198,87,488,320,87,488,220,16,15,15,8,284,220,16,16,15,198,14085,271,40582,124436,91406,1478,144073,126242,1447,14085,198,17,87,488,220,16,15,15,284
,220,16,16,15,198,14085,271,16206,26289,6949,2108,26289,68363,144073,126242,220,16,15,15,5805,26289,6949,21229,47162,5474,26289,127174,18943,1802,14
47,14085,198,17,87,284,220,16,15,198,14085,271,24110,26289,13073,126242,13073,220,17,1447,14085,198,87,284,220,20,198,14085,271,59527,58038,11,38133
,26289,14191,18108,28371,16748,140661,17787,400,220,20,400,140912,382,98574,32642,26289,2104,13132,510,12,126699,26289,14191,10885,1456,12150,130141
,25,400,379,284,856,488,220,16,15,15,284,220,20,400,220,16,15,15,284,220,16,15,20,400,140912,624,12,27490,130601,25,400,220,20,488,220,16,15,20,284,
220,16,16,15,400,140912,11,8839,231,1456,5805,26289,6949,132502,26289,30400,144073,13932,6442,6715,26289,133771,26289,382,334,16206,26289,6949,13250
2,26289,6949,4824,95518,48420,28371,13132,87415,12150,88248,144073,57960,79075,90,20,31716,140912,13},{"total_duration":163012349000,"load_duration":
8656580,"prompt_eval_count":63,"prompt_eval_cached_count":62,"prompt_eval_duration":117611000,"eval_count":1382,"eval_duration":162817597000}
C:\Users\oleks>
```

5. Результати Пата3.2:3b

Аналогічно було проведено п'ять запусків інструктивної моделі.

За результатами експерименту модель поведилася менш стабільно.

Наприклад, під час одного запуску вона правильно визначила ціну ручки як 5 грн, тоді як під час інших запусків спостерігалися неправильні відповіді, зокрема 100 грн та 110 грн.

За числовими показниками п'яти запусків отримано:

Таблиця 2.2 – Результати запусків Пата3.2:3b

№ запуску eval_count Загальний час, с

1	185	200,92
2	107	92,73
3	194	125,70
4	157	99,13

№ запуску eval_count Загальний час, с

5	117	75,56
---	-----	-------

Середнє	152,0	118,81
---------	-------	--------

Середня кількість токенів:

$(185 + 107 + 194 + 157 + 117) / 5 = 152$ токени

Середній час:

$(200,92 + 92,73 + 125,70 + 99,13 + 75,56) / 5 \approx 118,81$ с

Рисунки 8–12 – Результати п'яти запусків llama3.2:3b.

у llama3.2:3b.

Отже, у проведеному експерименті міркувальна модель генерувала приблизно:

$1250,6 / 152 \approx 8,23$ раз

більше токенів.

За загальним часом:

- qwen3:4b — 155,18 с;
- llama3.2:3b — 118,81 с.

Міркувальна модель у середньому витрачала приблизно на 36,37 с більше на один запуск.

Таблиця 2 – Задача на міркування (середнє з 5 запусків)

Показник	qwen3:4b	llama3.2:3b
Правильних із 5 не підтверджено всіма скрінами щонайменше 1 із 5		
eval_count	1250,6	152,0
Час, с	155,18	118,81

Примітка. Точне значення рядка «Правильних із 5» неможливо достовірно відновити з наданих скріншотів, оскільки на частині кадрів поле response не потрапило в область зображення. Тому для звіту коректніше не вигадувати це значення.

7. Відповіді на контрольні питання

1. Обидві моделі дали однакову відповідь на просте фактичне питання, але міркувальна модель згенерувала значно більше токенів. Куди пішли ці токени?

Додаткові токени qwen3:4b були витрачені переважно на внутрішню чернетку міркувань, яка повертається Ollama в полі thinking. Тому видима відповідь могла складатися лише з одного слова, хоча перед її формуванням модель згенерувала значно більший обсяг тексту.

2. На яких типах задач міркувальні моделі дають перевагу над інструктивними, а на яких їх використання не виправдане?

Міркувальні моделі доцільні для задач, які потребують послідовних логічних або обчислювальних кроків. Для простих фактичних запитів додаткове міркування може бути невиправданим, оскільки збільшує кількість токенів і загальний час відповіді без суттєвого покращення кінцевого результату.

3. Чим відрізняється процес навчання міркувальної моделі від інструктивної?

Інструктивні моделі орієнтовані на виконання інструкцій користувача та безпосереднє формування кінцевої відповіді. Міркувальні моделі додатково налаштовують на виконання проміжних етапів розв'язання складних задач перед формуванням остаточної відповіді.

4. Чи означає більша кількість згенерованих токенів у міркувальній моделі, що вона «розумніша» за інструктивну?

Ні. Кількість токенів характеризує обсяг генерації, але сама по собі не визначає правильність або якість результату. Наприклад, у фактичному питанні обидві моделі правильно відповіли «Київ», хоча qwen3:4b використала значно більше токенів.

5. Як пояснити різницю у швидкості генерації між qwen3:4b та llama3.2:3b?

Швидкість генерації залежить від характеристик і розміру моделі, способу її виконання та продуктивності обладнання. Крім того, загальний час відповіді залежить від кількості згенерованих токенів, тому модель із великим обсягом внутрішніх міркувань може виконувати запит довше.

Власні спостереження

Під час проведення експерименту було помітно, що міркувальна модель qwen3:4b витрачає значно більше токенів на формування відповіді. Особливо наочно це проявилось під час простого питання про столицю України: обидві моделі дали правильну відповідь, однак qwen3:4b додатково виконувала внутрішні міркування.

Під час задачі про ручку та зошит qwen3:4b генерувала в середньому 1250,6 токена, тоді як llama3.2:3b — лише 152 токени. Таким чином, міркувальна модель виконувала приблизно у 8,2 раза більший обсяг генерації.

Водночас результати llama3.2:3b показали, що швидша та коротша генерація не гарантує правильності. Під час повторних запусків однакового запиту модель могла отримувати різні результати.

На мою думку, міркувальні моделі доцільно використовувати для задач, де справді необхідний багатокроковий аналіз. Для простих фактичних запитів інструктивна модель може бути практичнішою, оскільки використовує менше tokenів і не витрачає ресурси на зайві внутрішні міркування.

Висновок

У ході виконання першої частини лабораторної роботи було встановлено Ollama та завантажено локальні мовні моделі qwen3:4b і llama3.2:3b. Було перевірено можливість взаємодії з моделями через локальний HTTP API та проаналізовано основні показники, які повертає Ollama.

У Завданні 1 було проведено порівняння міркувальної та інструктивної моделей. На простому фактичному питанні обидві моделі дали правильну відповідь, однак qwen3:4b використала 101 токен, тоді як llama3.2:3b — лише 5. Це показало, що міркувальна модель може виконувати значний обсяг додаткової генерації навіть тоді, коли задача не потребує складного аналізу.

Під час задачі на міркування qwen3:4b генерувала в середньому 1250,6 токена із середнім загальним часом 155,18 с. Для llama3.2:3b середня кількість становила 152 токени, а середній час — 118,81 с.

Отримані результати демонструють практичну «ціну роздумів»: внутрішнє міркування суттєво збільшує обсяг генерації та може збільшувати загальний час відповіді. При цьому більша кількість tokenів сама по собі не гарантує кращого результату. Тому вибір між міркувальною та інструктивною моделлю має залежати від складності задачі, необхідної швидкості та вимог до якості відповіді.

Крок 3. Керування моделлю з програми на Python

Для керування локальною мовною моделлю з програми було використано мову програмування Python та бібліотеку openai. Для ізоляції залежностей проєкту створено віртуальне середовище:

```
mkdir lab1
```

```
cd lab1
```

```
python -m venv venv
```

```
venv\Scripts\activate.bat
```

Після активації віртуального середовища було встановлено необхідні бібліотеки:

```
pip install openai python-dotenv
```

Встановлення завершилося успішно. Для взаємодії з локальним сервером Ollama було створено файл hello.py. Як адресу API використано `http://localhost:11434/v1`, що дозволяє працювати з Ollama через OpenAI-сумісний інтерфейс.

Рисунок 13-14 – Створення віртуального середовища та встановлення бібліотек.

```
C:\WINDOWS\system32\cmd.exe

C:\Users\oleks>mkdir lab1

C:\Users\oleks>cd lab1

C:\Users\oleks\lab1>python -m venv venv

C:\Users\oleks\lab1>venv\Scripts\activate.bat

(venv) C:\Users\oleks\lab1>pip install openai python-dotenv
Collecting openai
  Downloading openai-3.19.2-py3-none-any.whl.metadata (44 kB)
Collecting python-dotenv
  Downloading python_dotenv-1.2.3-py3-none-any.whl.metadata (29 kB)
Collecting anyio<5,>=4.10.0 (from openai)
  Downloading anyio-4.15.1-py3-none-any.whl.metadata (4.7 kB)
Collecting httpx<3,>=2.12.0 (from openai)
  Downloading httpx-2.13.1-py3-none-any.whl.metadata (9.8 kB)
Collecting jiter<1,>=0.16.0 (from openai)
  Downloading jiter-0.17.0-cp313-cp313-win_and64.whl.metadata (5.3 kB)
Collecting pydantic!=2.0.*,!=2.1.*,!=2.2.*,!=2.3.*,<3,>=1.10.13 (from openai)
  Downloading pydantic-2.13.5-py3-none-any.whl.metadata (110 kB)
Collecting sniffio (from openai)
  Downloading sniffio-1.3.1-py3-none-any.whl.metadata (3.9 kB)
Collecting typing-extensions<5,>=4.14 (from openai)
  Downloading typing_extensions-4.16.0-py3-none-any.whl.metadata (3.3 kB)
Collecting idna>=2.8 (from anyio<5,>=4.10.0->openai)
  Downloading idna-3.20-py3-none-any.whl.metadata (7.2 kB)
Collecting httpcore2==2.13.1 (from httpx<3,>=2.12.0->openai)
  Downloading httpcore2-2.13.1-py3-none-any.whl.metadata (26 kB)
Collecting truststore>=0.10 (from httpx<3,>=2.12.0->openai)
  Downloading truststore-0.10.4-py3-none-any.whl.metadata (4.4 kB)
Collecting h11>=0.16 (from httpcore2==2.13.1->httpx<3,>=2.12.0->openai)
  Downloading h11-0.16.0-py3-none-any.whl.metadata (8.3 kB)
Collecting annotated-types>=0.6.0 (from pydantic!=2.0.*,!=2.1.*,!=2.2.*,!=2.3.*,<3,>=1.10.13->openai)
  Downloading annotated_types-0.8.0-py3-none-any.whl.metadata (15 kB)
Collecting pydantic-core==2.46.5 (from pydantic!=2.0.*,!=2.1.*,!=2.2.*,!=2.3.*,<3,>=1.10.13->openai)
  Downloading pydantic_core-2.46.5-cp313-cp313-win_and64.whl.metadata (6.7 kB)
Collecting typing-inspection>=0.4.2 (from pydantic!=2.0.*,!=2.1.*,!=2.2.*,!=2.3.*,<3,>=1.10.13->openai)
  Downloading typing_inspection-0.4.4-py3-none-any.whl.metadata (2.6 kB)
Downloading openai-3.19.2-py3-none-any.whl (2.1 MB)
----- 2.1/2.1 MB 18.7 MB/s 0:00:00

C:\WINDOWS\system32\cmd.exe
Downloading idna-3.20-py3-none-any.whl (69 kB)
Downloading truststore-0.10.4-py3-none-any.whl (18 kB)
Downloading typing_inspection-0.4.4-py3-none-any.whl (14 kB)
Downloading sniffio-1.3.1-py3-none-any.whl (10 kB)
Installing collected packages: typing-extensions, truststore, sniffio, python-dotenv, jiter, idna, h11, annotated-types, typing-inspection, pydantic-core, httpcore2, anyio, pydantic, httpx, openai
Successfully installed annotated-types-0.8.0 anyio-4.15.1 h11-0.16.0 httpcore2-2.13.1 httpx-2.13.1 idna-3.20 jiter-0.17.0 openai-3.19.2 pydantic-2.13.5 pydantic-core-2.46.5 python-dotenv-1.2.3 sniffio-1.3.1 truststore-0.10.4 typing-extensions-4.16.0 typing-inspection-0.4.4

(venv) C:\Users\oleks\lab1>notepad hello.py

(venv) C:\Users\oleks\lab1>python hello.py
У Python існують дві основні структури даних для роботи з даними: списки та кортежі.

**Списки**
Список – це набір елементів, которые могут быть числами, словами, символами, тощо. Списки виявляються зазначенням відкритого круга з позначенням елемента, який зображується після коми. Багаторазова елементна операція для над списком здійснюється за допомогою методу "append()", який зазначає новий елемент за допомогою ключового слова "append()".

Наприклад:
'''python
my_list = []
my_list.append(1)
my_list.append(2)
my_list.append(3)
print(my_list) # [1, 2, 3]
'''

**Кортежі**
Кортеж – єдиний тип елементного об'єднання в Python, який виявляється зазначенням круглих скобок з елементами. Кортежі зазначаються з допомогою круглих скобок з елементами. Кортеж не може бути змінним, адже його елементи зазначаються в його елементах. Багаторазова елементна операція для над кортежем здійснюється за допомогою методу "index()" зазначення позиції елемента, який здійснюється за допомогою ключового слова "index()".

Наприклад:
'''python
my_tuple = (1, 2, 3)
print(my_tuple) # (1, 2, 3)
# my_tuple[0] = 5 # Err! cannot assign to tuple element
'''

В цілому, список є більш динамічною структурою даних, тоді як кортеж є більш стабільною структурою даних.
CompletionUsage(completion_tokens=396, prompt_tokens=54, total_tokens=450, completion_tokens_details=None, prompt_tokens_details=None, audio_tokens=None, cache_write_tokens=None, cached_tokens=0, image_tokens=None, text_tokens=None)
```

Крок 3. Керування моделлю з програми на Python

На цьому етапі було налаштовано програмне середовище для керування локальною мовною моделлю Ollama за допомогою Python та OpenAI-сумісного API.

1. Перевірка Python

Спочатку було перевірено наявність Python у системі за допомогою команди:

```
python --version
```

Для виконання лабораторної роботи необхідно використовувати Python версії 3.8 або новішої.

2. Створення папки проєкту та віртуального середовища

Для лабораторної роботи було створено окрему папку lab1:

```
mkdir lab1
```

```
cd lab1
```

Після цього створено віртуальне середовище:

```
python -m venv venv
```

Для його активації виконано:

```
venv\Scripts\activate.bat
```

Після успішної активації на початку командного рядка з'явилася позначка:

```
(venv) C:\Users\oleks\lab1>
```

Це підтвердило, що віртуальне середовище було створено та активовано правильно.

3. Встановлення необхідних бібліотек

Для роботи з OpenAI-сумісним API було встановлено бібліотеки openai та python-dotenv:

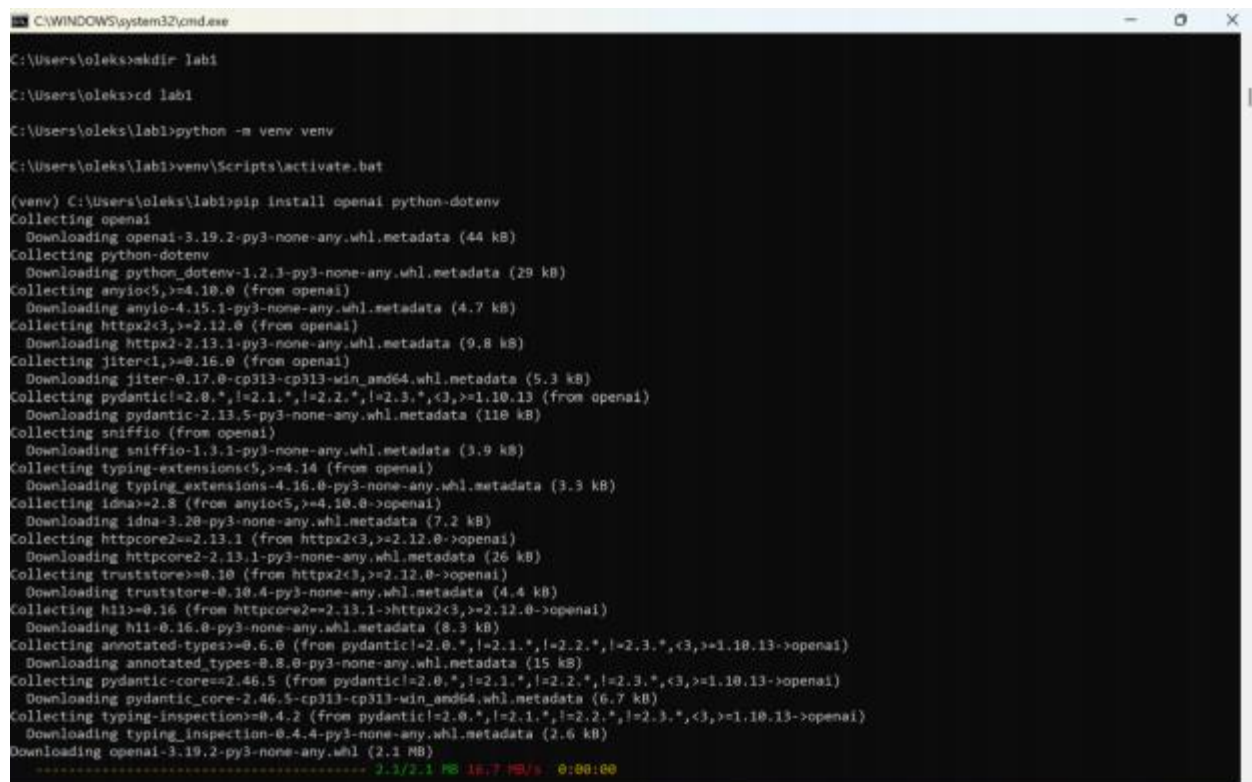
```
pip install openai python-dotenv
```

Встановлення завершилося успішно, про що свідчило повідомлення
Successfully installed.

Бібліотека openai в цій роботі використовується як клієнт для стандартного формату API-запитів . При цьому запити надсилаються не до хмарної моделі OpenAI, а до локального сервера Ollama.

Використання OpenAI-сумісного API дозволяє застосовувати подібний програмний код для різних мовних моделей, змінюючи адресу сервера та модель.

Рисунок 13 – Створення віртуального середовища та встановлення бібліотек .



```
C:\WINDOWS\system32\cmd.exe
C:\Users\oleks>mkdir lab1
C:\Users\oleks>cd lab1
C:\Users\oleks\lab1>python -m venv venv
C:\Users\oleks\lab1\venv\Scripts\activate.bat
(venv) C:\Users\oleks\lab1>pip install openai python-dotenv
Collecting openai
  Downloading openai-3.19.2-py3-none-any.whl.metadata (44 kB)
Collecting python-dotenv
  Downloading python_dotenv-1.2.1-py3-none-any.whl.metadata (29 kB)
Collecting anyio<5,>=4.10.0 (from openai)
  Downloading anyio-4.15.1-py3-none-any.whl.metadata (4.7 kB)
Collecting httpx2<3,>=2.12.0 (from openai)
  Downloading httpx2-2.13.1-py3-none-any.whl.metadata (9.8 kB)
Collecting jiter<1,>=0.16.0 (from openai)
  Downloading jiter-0.17.0-cp313-cp313-win_and64.whl.metadata (5.3 kB)
Collecting pydantic<2.0.*,!=2.1.*,!=2.2.*,!=2.3.*,<3,>=1.10.13 (from openai)
  Downloading pydantic-2.13.5-py3-none-any.whl.metadata (110 kB)
Collecting sniffio (from openai)
  Downloading sniffio-1.3.1-py3-none-any.whl.metadata (3.9 kB)
Collecting typing-extensions<5,>=4.14 (from openai)
  Downloading typing_extensions-4.16.0-py3-none-any.whl.metadata (3.3 kB)
Collecting idna>=2.8 (from anyio<5,>=4.10.0->openai)
  Downloading idna-3.20-py3-none-any.whl.metadata (7.2 kB)
Collecting httpcore2<=2.13.1 (from httpx2<3,>=2.12.0->openai)
  Downloading httpcore2-2.13.1-py3-none-any.whl.metadata (26 kB)
Collecting truststore>=0.10 (from httpx2<3,>=2.12.0->openai)
  Downloading truststore-0.10.4-py3-none-any.whl.metadata (4.4 kB)
Collecting h11>=0.16 (from httpcore2<=2.13.1->httpx2<3,>=2.12.0->openai)
  Downloading h11-0.16.0-py3-none-any.whl.metadata (8.3 kB)
Collecting annotated-types>=0.6.0 (from pydantic<2.0.*,!=2.1.*,!=2.2.*,!=2.3.*,<3,>=1.10.13->openai)
  Downloading annotated_types-0.6.0-py3-none-any.whl.metadata (15 kB)
Collecting pydantic-core==2.46.5 (from pydantic<2.0.*,!=2.1.*,!=2.2.*,!=2.3.*,<3,>=1.10.13->openai)
  Downloading pydantic_core-2.46.5-cp313-cp313-win_and64.whl.metadata (6.7 kB)
Collecting typing-inspection>=0.4.2 (from pydantic<2.0.*,!=2.1.*,!=2.2.*,!=2.3.*,<3,>=1.10.13->openai)
  Downloading typing_inspection-0.4.4-py3-none-any.whl.metadata (2.6 kB)
Downloading openai-3.19.2-py3-none-any.whl (2.1 MB)
2.1/2.1 MB 16.7 MB/s 0:00:00
```

4. Створення програми hello.py

Для програмного звернення до локальної моделі було створено файл

hello.py. Для його створення виконано:

notepad hello.py

У файл було записано програму:

```
from openai import OpenAI
```

```
# Кажемо клієнту стукати не в OpenAI, а на наш комп'ютер
```

```
client = OpenAI(
```

```
    base_url="http://localhost:11434/v1",
```

```

    api_key="ollama", # Ollama ключ не перевіряє, але поле обов'язкове
)

response = client.chat.completions.create(
    model="qwen3:4b",
    messages=[
        {
            "role": "system",
            "content": "Ти лаконічний технічний асистент."
        },
        {
            "role": "user",
            "content": "Поясни різницю між списком і кортежем у Python."
        },
    ],
    temperature=0.7,
)

print(response.choices[0].message.content)
print(response.usage)

```

У програмі створюється клієнт OpenAI, якому в параметрі:

`base_url="http://localhost:11434/v1"` передається адреса локального сервера Ollama.

Поле:

`api_key="ollama"`

є обов'язковим для клієнта, хоча локальна Ollama цей ключ не перевіряє.

Для отримання тексту відповіді використовується:

```
print(response.choices[0].message.content)
```

Для отримання інформації про використані токени:

```
print(response.usage)
```

`response.usage` містить такі основні показники:

- `prompt_tokens` — кількість токенів вхідного запиту;
- `completion_tokens` — кількість токенів генерації;
- `total_tokens` — загальна кількість використаних токенів.

Створену програму було запущено командою:

```
python hello.py
```

У результаті було підтверджено, що локальною моделлю Ollama можна керувати безпосередньо з програми мовою Python через OpenAI-сумісний API.

Рисунок 14 – Результат запуску програми `hello.py`.

```
(venv) C:\Users\oleks\lab1\python hello.py
У Python існують дві основні структури даних для роботи з даними: списки та кортежі.

**Списки**
Список - это набор элементов, которые могут быть числами, словами, символами, тощо. Списки выявляются зазначенням відкритого круга з позначенням елемента, який зображується після черви. Багаторазна елементна оперативна дія над списком здійснюється за допомогою методу 'append()', який зазначається за допомогою ключового слова 'append()'.

Наприклад:
... python
my_list = []
my_list.append(1)
my_list.append(2)
my_list.append(3)
print(my_list) # [1, 2, 3]
...

**Кортежі**
Кортеж - єдиний тип елементного об'єднання в Python, який виявляється зазначенням круглих скобок з елементами. Кортежі зазначаються з допомогою круглих скобок з елементами. Кортеж не може бути змінним, адже його елементи зазначаються в його елементах. Багаторазна елементна оперативна дія над кортежем здійснюється за допомогою методу 'index()' зазначення позиції елемента, який здійснюється за допомогою ключового слова 'index()'.

Наприклад:
... python
my_tuple = (1, 2, 3)
print(my_tuple) # (1, 2, 3)
# my_tuple[0] = 5 # Err! cannot assign to tuple element
...

В цілому, список є більш динамічною структурою даних, тоді як кортеж є більш стабільною структурою даних.
CompletionUsage(completion_tokens=396, prompt_tokens=54, total_tokens=450, completion_tokens_details=None, prompt_tokens_details=PromptTokensDetails(audio_tokens=None, cache_write_tokens=None, cached_tokens=0, image_tokens=None, text_tokens=None))
```

Висновок до Кроку 3

У ході виконання Кроку 3 було створено папку проєкту та віртуальне середовище Python, встановлено бібліотеки openai і python-dotenv та створено програму hello.py.

За допомогою OpenAI-сумісного API було реалізовано програмне звернення до локального сервера Ollama. Програма дозволяє отримувати відповідь мовної моделі та статистику використаних токенів, що необхідно для подальших експериментів.

Завдання 2. Роль system-повідомлення: керування поведінкою моделі

Мета завдання

Дослідити вплив system-повідомлення на поведінку мовної моделі та порівняти, наскільки моделі llama3.2:3b і qwen3:4b дотримуються інструкцій щодо мови, стилю та довжини відповіді.

Модель не «пам'ятає» попередню розмову автоматично. При кожному виклику необхідний контекст передається їй у параметрі messages.

Використовуються такі ролі повідомлень:

- system — інструкція, яка задає поведінку моделі: мову, стиль, довжину та формат відповіді;
- user — повідомлення користувача;

- assistant — попередня відповідь моделі, яка може передаватися назад для продовження діалогу.

Практичне значення system-повідомлення полягає в тому, що поведінкою моделі можна керувати без її донавчання.

Варіант завдання

Для варіанта №1 використано питання:

«Поясни різницю між списком і кортежем у Python.»

Експеримент було проведено з двома моделями:

- llama3.2:3b;
- qwen3:4b.

Для кожної моделі використано два system-повідомлення:

Ти лаконічний технічний асистент.

та:

Відповідай лише українською, максимум двома реченнями.

Таким чином, загалом було виконано чотири запуски.

Крок 1. Перевірка llama3.2:3b з першим system-повідомленням

Для першого запуску використано програму:

```
from openai import OpenAIclient = OpenAI(
base_url="http://localhost:11434/v1", api_key="ollama",)SYSTEM = "Ти
лаконічний технічний асистент."USER = "Поясни різницю між списком і
кортежем у Python."response = client.chat.completions.create(
model="llama3.2:3b", messages=[ {"role": "system", "content": SYSTEM},
{"role": "user", "content": USER}, ],
temperature=0.7,)print(response.choices[0].message.content)print(response.usage)
```

Програму запущено командою:

```
python hello.py
```

Модель сформувала досить велику відповідь із поясненням списків і кортежів та прикладами програмного коду.

У відповіді переважала українська мова, однак зустрічалися російські слова та конструкції. Крім того, відповідь була досить великою, хоча system-повідомлення вимагало бути лаконічним.

Було отримано:

```
prompt_tokens = 54
```

```
completion_tokens = 396
```

```
total_tokens = 450
```

Отже, вимогу щодо лаконічності модель виконала лише частково.

Рисунок 15 – Результат першого запуску llama3.2:3b.



```
(venv) C:\Users\oleks\lab1>python hello.py
Списки (lists) – зірки з чергуванням елементами, а кортежи (tuples) – зірки з фіксованими елементами.
CompletionUsage(completion_tokens=42, prompt_tokens=58, total_tokens=100, completion_tokens_details=None, prompt_tokens_details=PromptTokensDetails(audio_tokens=None, cache_write_tokens=None, cached_tokens=0, image_tokens=None, text_tokens=None))
```

Крок 2. Перевірка llama3.2:3b з другим system-повідомленням

System-повідомлення було змінено на:

SYSTEM = "Відповідай лише українською, максимум двома реченнями."

Після повторного запуску модель сформувала коротку відповідь:

«Списки (lists) – зірки з чергуванням елементами, а кортежи (tuples) – зірки з фіксованими елементами.»

Відповідь складалася з одного речення, тому обмеження «максимум двома реченнями» було виконано. Водночас у тексті присутні мовні неточності.

Отримано:

```
prompt_tokens = 58
```

```
completion_tokens = 42
```

```
total_tokens = 100
```

Після конкретизації system-повідомлення кількість completion_tokens зменшилася з 396 до 42.

Рисунок 16 – Результат llama3.2:3b з обмеженням до двох речень.

```
(venv) C:\Users\oleks\lab1\python hello.py
Списки (lists) – зірки з чергуванням елементами, а кортежі (tuples) – зірки з фіксованими елементами.
CompletionUsage(completion_tokens=42, prompt_tokens=55, total_tokens=100, completion_tokens_details=None, prompt_tokens_details=None,
audio_tokens=None, cache_write_tokens=None, cached_tokens=0, image_tokens=None, text_tokens=None))
```

Крок 3. Перевірка qwen3:4b з першим system-повідомленням

Наступний експеримент було проведено з моделлю:

model="qwen3:4b"

System-повідомлення:

SYSTEM = "Ти лаконічний технічний асистент."

Модель сформулила українською мовою структуровану відповідь, у якій навела основні відмінності між списком і кортежем.

Незважаючи на відносно невеликий обсяг видимої відповіді, було отримано:

prompt_tokens = 55

completion_tokens = 1909

total_tokens = 1964

Таким чином, qwen3:4b використала значно більше вихідних токенів, ніж llama3.2:3b.

Це пов'язано з тим, що qwen3:4b є міркувальною моделлю та може використовувати частину вихідних токенів для внутрішнього процесу міркування.

Рисунок 17 – Результат першого запуску qwen3:4b.

```
(venv) C:\Users\oleks\lab1\python hello.py
Різниця між списком і кортежем у Python:

1. **Мутабельність**: Списки – **мутабельні** (можна змінювати елементи), кортежі – **немутабельні** (незмінні).
2. **Синтаксис**: Список в '[ ]', кортеж в '()'.
3. **Ключі в словниках**: Кортежі можуть бути ключами (хеш-повні), списки – ні.

Кортежі швидше для доступу до елементів через індекс.
CompletionUsage(completion_tokens=1909, prompt_tokens=55, total_tokens=1964, completion_tokens_details=None, prompt_tokens_details=None,
audio_tokens=None, cache_write_tokens=None, cached_tokens=0, image_tokens=None, text_tokens=None))
```

Крок 4. Перевірка qwen3:4b з другим system-повідомленням

System-повідомлення було змінено на:

SYSTEM = "Відповідай лише українською, максимум двома реченнями."

Модель сформувала відповідь:

«Список є мутабільним, тому можна змінювати його елементи, а кортеж — незмінним, що забороняє будь-які модифікації після створення. Кортєжі швидкіші для ітерації та вимагають менше пам'яті, але використовуються для фіксованих наборів даних.»

Відповідь була сформована українською мовою та складалася рівно з двох речень. Отже, модель виконала вимогу system-повідомлення.

Отримано:

prompt_tokens = 64

completion_tokens =

1378 total_tokens = 1442

Незважаючи на те, що видима відповідь складалася лише з двох речень, qwen3:4b використала 1378 completion-токенів.

Рисунок 18 – Результат qwen3:4b з обмеженням до двох речень.

```
(venv) C:\Users\oleks\lab1>python hello.py
Список є мутабільним, тому можна змінювати його елементи, а кортеж – незмінним, що забороняє будь-які модифікації після створення. Кортєжі швидкіші
для ітерації та вимагають менше пам'яті, але використовуються для фіксованих наборів даних.
CompletionUsage(completion_tokens=1378, prompt_tokens=64, total_tokens=1442, completion_tokens_details=None, prompt_tokens_details=PromptTokensDetail
s(audio_tokens=None, cache_write_tokens=None, cached_tokens=3, image_tokens=None, text_tokens=None))
```

Крок 5. Заповнення таблиць

Таблиця 3 – Вплив system-повідомлення

Модель	System-повідомлення	Довжина відповіді, символів	Мова відповіді	Кількість речень	Інструкцію виконано?
llama3.2:3b	Лаконічний асистент	не визначено	переважно українська, є російські фрагменти	багато	частково
llama3.2:3b	Українською, максимум 2 речення	не визначено	переважно українська, є мовні неточності	1	частково
qwen3:4b	Лаконічний асистент	не визначено	українська	кілька	так
qwen3:4b	Українською, максимум 2 речення	не визначено	українська	2	так

Точну кількість символів доцільно визначати програмно, оскільки підрахунок зі скріншота може дати неточний результат. Для цього до програми можна додати:

```
text = response.choices[0].message.contentprint("Кількість символів:", len(text))
```

Таблиця 4 – Витрати токенів

Модель	System-повідомлення	prompt_tokens	completion_tokens	total_tokens
llama3.2:3b	Лаконічний асистент	54	396	450
llama3.2:3b	Українською, максимум 2 речення	58	42	100
qwen3:4b	Лаконічний асистент	55	1909	1964
qwen3:4b	Українською, максимум 2 речення	64	1378	1442

Крок 6. Відповіді на питання

1. Яка модель точніше дотрималась обмеження «максимум двома реченнями»?

Обидві моделі виконали обмеження: Llama3.2:3b сформувала одне речення, а qwen3:4b — рівно два. При цьому відповідь qwen3:4b була більш природною українською мовою.

2. Чи були випадки, коли модель відповіла не українською, попри пряму вказівку?

Повного переходу на іншу мову після прямої вказівки не було. Проте у відповіді Llama3.2:3b спостерігалися мовні неточності, тоді як qwen3:4b сформувала відповідь українською.

3. Як змінився `prompt_tokens` при зміні `system`-повідомлення і чому?

Для Llama3.2:3b значення збільшилося з 54 до 58, а для qwen3:4b — з 55 до 64. Це відбулося через зміну `system`-повідомлення, оскільки його текст також є частиною вхідного контексту та враховується в `prompt_tokens`.

4. Чому в qwen3:4b значення `completion_tokens` значно більше, ніж у Llama3.2:3b, навіть при однаковій довжині видимої відповіді?

qwen3:4b є міркувальною моделлю і може генерувати додаткові внутрішні міркування перед формуванням кінцевої відповіді. Через це кількість `completion_tokens` залишається великою навіть при короткій видимій відповіді.

Крок 7. Аналіз результатів за допомогою чат-бота

1. Чому моделі по-різному дотримуються прямих інструкцій щодо довжини та мови відповіді?

Мовні моделі можуть відрізнятися способом навчання та налаштування на виконання інструкцій. Тому одна модель може точніше виконувати вимоги щодо мови й формату, тоді як інша може частково їх порушувати або допускати мовні помилки.

2. Чим `system`-повідомлення відрізняється від `user`-повідомлення з точки зору моделі — чи справді воно має більший пріоритет?

`system`-повідомлення задає загальні правила поведінки моделі, а `user`-повідомлення містить конкретний запит користувача. `System`-інструкція

призначена для керування поведінкою моделі протягом усієї взаємодії, хоча конкретна модель може не завжди ідеально її виконувати.

3. Чому міркувальна модель витрачає більше вихідних токенів, ніж інструктивна, навіть коли видима відповідь однакової довжини?

Міркувальна модель може виконувати додаткову внутрішню генерацію перед формуванням остаточної відповіді. У проведеному експерименті коротка відповідь qwen3:4b використала 1378 completion-токенів, тоді як коротка відповідь llama3.2:3b — лише 42.

4. Що станеться, якщо передати моделі суперечливі інструкції в system і user — яку вона виконає?

System-повідомлення призначене для встановлення загальних правил, у межах яких має виконуватися запит користувача. Однак мовна модель не є жорсткою системою правил, тому особливо невеликі моделі можуть не завжди стабільно вирішувати конфлікт між інструкціями.

5. Чому малі мовні моделі часто гірше дотримуються інструкцій щодо мови відповіді, ніж великі?

Малі моделі мають обмеженіші можливості одночасно враховувати зміст запиту та декілька додаткових вимог. Це може проявлятися у змішуванні мов, граматичних помилках або частковому недотриманні заданого формату.

Власні думки щодо ролі system-повідомлення

Проведений експеримент показав, що system-повідомлення суттєво впливає на поведінку мовної моделі. Після додавання конкретної вимоги відповідати лише українською мовою максимум двома реченнями обидві моделі сформували значно коротші видимі відповіді.

Особливо помітною була зміна для llama3.2:3b: кількість completion_tokens зменшилася з 396 до 42, а total_tokens — з 450 до 100. Для qwen3:4b кількість completion_tokens зменшилася з 1909 до 1378, але залишилася значно більшою, ніж у llama3.2:3b.

На мою думку, system-повідомлення є зручним способом керування мовою, стилем та довжиною відповіді, але воно не гарантує абсолютного виконання

інструкції. Якщо програма вимагає строгого формату, отриманий результат доцільно додатково перевіряти програмно. Наприклад, можна контролювати кількість символів або речень, перевіряти формат JSON та повторювати запит, якщо результат не відповідає заданим вимогам.

Висновок

У ході виконання Кроку 3 було налаштовано Python-середовище для програмної взаємодії з локальними мовними моделями. Було створено віртуальне середовище, встановлено бібліотеки `openai` та `python-dotenv` і реалізовано програму `hello.py`, яка звертається до локального сервера Ollama через OpenAI-сумісний API.

У Завданні 2 було досліджено вплив `system`-повідомлення на поведінку моделей `llama3.2:3b` та `qwen3:4b`. Для кожної моделі було використано два `system`-повідомлення: загальну вимогу бути лаконічним технічним асистентом і конкретну вимогу відповідати лише українською мовою максимум двома реченнями.

Результати показали, що конкретизація `system`-повідомлення суттєво впливає на довжину відповіді. `llama3.2:3b` після зміни інструкції сформувала одне речення, а `qwen3:4b` — два речення.

Також було виявлено значну різницю у витратах токенів. При обмеженні до двох речень `llama3.2:3b` використала 42 completion-токени, тоді як `qwen3:4b` — 1378. Це демонструє особливість міркувальної моделі, яка може використовувати значну кількість токенів для внутрішньої генерації навіть при короткій видимій відповіді.

Отже, `system`-повідомлення є ефективним способом керування поведінкою мовної моделі, однак для отримання строго визначеного формату самої текстової інструкції може бути недостатньо. У практичних застосунках доцільно поєднувати `system`-інструкції з програмною перевіркою отриманого результату.

Крок 4. Підключення хмарної моделі

На цьому етапі було виконано підключення хмарної мовної моделі та проведено порівняння її роботи з локальною моделлю. OpenAI-сумісний

формат API дозволяє використовувати практично однаковий програмний код для локального та хмарного провайдера, змінюючи адресу сервера, API-ключ та назву моделі.

Для локальної моделі використовувався сервер Ollama, а для хмарної — API провайдера Groq.

1. Отримання API-ключа

Для доступу до хмарної моделі було створено API-ключ провайдера Groq.

API-ключ використовується для авторизації програмного запиту на сервері провайдера. Оскільки ключ надає доступ до облікового запису та його лімітів, його не можна зберігати безпосередньо у програмному коді або публікувати у відкритому репозиторії.

Для роботи використовувалась адреса:

<https://api.groq.com/openai/v1>

Спочатку передбачалося використати модель llama-3.3-70b-versatile, наведену в методичних матеріалах, однак під час запуску сервер повернув помилку 404 model_not_found. Тому для виконання експерименту було використано доступну хмарну модель:

openai/gpt-oss-120b

2. Збереження API-ключа

У папці проєкту було створено файл .env, у якому API-ключ зберігається окремо від програмного коду:

```
CLOUD_API_KEY=API_КЛЮЧ
```

Для запобігання випадковому додаванню ключа та віртуального середовища до Git було створено файл .gitignore:

```
.env
```

```
venv/
```

Таким чином, конфіденційні дані зберігаються окремо від вихідного коду програми.

Завдання 3. Локальна vs хмарна модель: єдиний інтерфейс

Мета завдання

Метою завдання було порівняти роботу локальної та хмарної мовних моделей, використовуючи єдиний OpenAI-сумісний програмний інтерфейс.

Для експерименту використано:

Локальна модель:

llama3.2:3b

Хмарна модель:

openai/gpt-oss-120b

Відповідно до варіанта №1 моделям було поставлено однакове питання:

«Поясни різницю між списком і кортежем у Python. Коротко.»

Крок 1. Налаштування провайдерів

Для зберігання параметрів локального та хмарного провайдерів було створено файл providers.py.

Код програми:

```
import os

from openai import OpenAI
from dotenv import load_dotenv

load_dotenv()

PROVIDERS = {
    "local": {
        "base_url": "http://localhost:11434/v1",
        "api_key": "ollama",
        "model": "llama3.2:3b",
    },
    "cloud": {
        "base_url": "https://api.groq.com/openai/v1",
```

```
        "api_key": os.getenv("CLOUD_API_KEY"),
        "model": "openai/gpt-oss-120b",
    },
}
```

```
def make_client(name):
    cfg = PROVIDERS[name]

    client = OpenAI(
        base_url=cfg["base_url"],
        api_key=cfg["api_key"]
    )

    return client, cfg["model"]
```

У словнику PROVIDERS зібрано всі параметри, які відрізняються між локальним та хмарним провайдером.

Для локальної моделі використовується адреса:

<http://localhost:11434/v1>

Для хмарної:

<https://api.groq.com/openai/v1>

API-ключ хмарного провайдера отримується з .env за допомогою:

```
os.getenv("CLOUD_API_KEY")
```

Тому конфіденційний ключ не записується безпосередньо у вихідному коді.

Крок 2. Створення програми порівняння

Для виконання однакового запиту до двох моделей було створено файл `compare.py`:

```
import time

from providers import make_client

QUESTION = "Поясни різницю між списком і кортежем у Python. Коротко."

for name in ["local", "cloud"]:
    client, model = make_client(name)

    t0 = time.perf_counter()

    response = client.chat.completions.create(
        model=model,
        messages=[
            {
                "role": "user",
                "content": QUESTION
            }
        ],
        temperature=0.7,
    )
```

```
elapsed = time.perf_counter() - t0

text = response.choices[0].message.content

print(f"==== {name.upper()} ({model}) =====")

print(text)

print(f"Час: {elapsed:.2f} c")

print(f"Довжина відповіді: {len(text)} символів")

print(response.usage)

print()
```

Однаковий цикл використовується для обох моделей. Функція `client.chat.completions.create()` викликається однаково, а відмінності між провайдерами знаходяться у файлі `providers.py`.

До програми також було додано:

```
print(f"Довжина відповіді: {len(text)} символів")
```

Це дозволило автоматично визначити довжину відповіді, необхідну для заповнення таблиці.

Крок 3. Запуск програми

Програму було запущено командою:

```
python compare.py
```

Під час виконання однаковий запит спочатку було надіслано локальній моделі llama3.2:3b, а потім хмарній моделі openai/gpt-oss-120b.

Локальна модель сформувала коротку відповідь, у якій пояснила основну відмінність: список є змінюваною структурою даних, а кортеж — незмінною. Також було наведено приклади методів списку append, insert та remove.

Хмарна модель надала значно детальнішу структуровану відповідь. Крім змінюваності, вона описала синтаксис, призначення, продуктивність, доступні методи та типові випадки використання списків і кортежів.

За результатами запуску локальна модель витратила 8,18 с, а хмарна — 1,23 с.

Рисунок 19 – Результат порівняння локальної моделі llama3.2:3b та хмарної моделі openai/gpt-oss-120b.

```
(venv) C:\Users\oleks\lab1\python compare.py
===== LOCAL (llama3.2:3b) =====
Різниця між списком і кортежем в Python:

- Список ('list') - набір елементів, які можуть бути змінені (додано, видалено, змінено) за допомогою методів such як 'append', 'insert', 'remove'.
- Кортеж ('tuple') - набір елементів, які не можуть бути змінені (додано, видалено, змінено).
Час: 8.18 с
Довжина відповіді: 283 символів
CompletionUsage(completion_tokens=92, prompt_tokens=47, total_tokens=139, completion_tokens_details=None, prompt_tokens_details=PromptTokensDetails(audio_tokens=None, cache_write_tokens=None, cached_tokens=46, image_tokens=None, text_tokens=None))

===== CLOUD (openai/gpt-oss-120b) =====
**Список ('list') vs. кортеж ('tuple') у Python**

|-----|
| **Список** ('list') | **Кортеж** ('tuple') | |
|---|---|---|
| **Змінюваність** | змінюваний (можна додавати, видаляти, змінювати елементи) | незмінний (після створення елементи не можна змінювати) |
| **Синтаксис** | '[...]' (наприклад, '[1, 2, 3]') | '(...)' або без дужок з комою (наприклад, '(1, 2, 3)' або '1, 2, 3') |
| **Призначення** | підходить для колекцій, які будуть змінюватись під час виконання | використовується для фіксованих наборів даних, ключів у словниках, швидкого доступу |
| **Продуктивність** | трохи повільніший через додаткові операції зі зміною | швидкий і споживає менше пам'яті, оскільки не підтримує зміни |
| **Методи** | має багато методів ('append', 'extend', 'pop', 'remove', 'sort' тощо) | має лише обмежений набір методів ('count', 'index') |
| **Використання** | коли потрібна динаміка (додавання/видалення елементів) | коли дані константні або використовуються як ключі у 'dict'/елементі 'set' |

**Коротко:**
- **Список** — змінюваний, гнучкий, підходить для динамічних колекцій.
- **Кортеж** — незмінний, більш ефективний за пам'яттю та швидкість, використовується для фіксованих даних.
Час: 1.23 с
Довжина відповіді: 1285 символів
CompletionUsage(completion_tokens=482, prompt_tokens=93, total_tokens=575, completion_tokens_details=CompletionTokensDetails(accepted_prediction_tokens=None, audio_tokens=None, reasoning_tokens=29, rejected_prediction_tokens=None, text_tokens=None), prompt_tokens_details=None, queue_time=0.04967)
```

Крок 4. Заповнення таблиці

Таблиця 5 – Порівняння локальної та хмарної моделі

Показник	Локальна	Хмарна
Назва моделі	llama3.2:3b	openai/gpt-oss-120b
Час відповіді, с	8,18	1,23

Показник	Локальна	Хмарна
completion_tokens	92	482
Довжина відповіді, символів	283	1285
Мова відповіді	українська з англійськими технічними термінами	українська з англійськими технічними термінами
Фактичних помилок у відповіді	суттєвих не виявлено	суттєвих не виявлено

Локальна модель використала 47 prompt_tokens, 92 completion_tokens та загалом 139 total_tokens.

Хмарна модель використала 93 prompt_tokens, 482 completion_tokens та загалом 575 total_tokens. У статистиці хмарної моделі також було зазначено reasoning_tokens=29.

Хмарна модель сформувала приблизно у 4,5 раза більшу за кількістю символів відповідь, але при цьому відповіла приблизно у 6,7 раза швидше за локальну.

Крок 5. Відповіді на питання

1. Скільки рядків коду довелося змінити, щоб перейти з локальної моделі на хмарну?

Основна логіка програми не змінювалася. Для іншого провайдера достатньо задати інші значення base_url, api_key та model, а функція client.chat.completions.create() використовується однаково для обох моделей.

2. Яка відповідь повніша й точніша за змістом? Наведіть конкретний приклад різниці.

У проведеному експерименті хмарна модель сформувала повнішу відповідь. Локальна модель переважно пояснила змінюваність списків і незмінюваність кортежів, тоді як хмарна додатково описала синтаксис, доступні методи,

використання кортежів як ключів словника та відмінності у використанні пам'яті.

3. Чи були в локальній моделі фактичні помилки або проблеми з українською мовою?

У фінальному запуску суттєвих фактичних помилок у локальній моделі не було виявлено: основну відмінність між списком і кортежем вона пояснила правильно. У відповіді використовувалися англійські технічні терміни `list`, `tuple`, `append`, `insert`, `remove`, але основний текст був українською.

4. Хмарна модель відповіла швидше за локальну — за рахунок чого, якщо запит летить через інтернет?

Хмарні моделі виконуються на потужній серверній інфраструктурі, оптимізованій для швидкого виконання нейромережових обчислень. У цьому експерименті час мережевої передачі виявився значно меншим за виграш у швидкості обчислень: хмарна модель відповіла за 1,23 с, тоді як локальний знадобилося 8,18 с.

Крок 6. Питання до чат-бота

1. Чому формат запитів OpenAI став стандартом, який підтримують інші провайдери, зокрема локальний сервер Ollama?

OpenAI-сумісний формат набув широкого поширення, оскільки багато програм та бібліотек уже використовують такий спосіб взаємодії з мовними моделями. Підтримка сумісного API дозволяє провайдерам спростити перенесення програм між різними моделями та сервісами.

2. Я порівнював локальну модель на 3 млрд параметрів і значно більшу хмарну модель. Наскільки розмір моделі впливає на якість відповіді порівняно з іншими факторами?

Кількість параметрів може впливати на можливості моделі, але сама по собі не визначає якість відповіді. Важливими також є навчальні дані, архітектура, методи донавчання, оптимізація та особливості конкретного завдання.

У цьому експерименті більша хмарна модель сформувала значно детальнішу відповідь: 1285 символів проти 283 символів у локальній.

3. Хмарна модель відповіла швидше за локальну, попри затримку мережі. Чим це пояснюється?

Хмарні провайдери використовують спеціалізоване високопродуктивне обладнання та оптимізоване програмне забезпечення для інференсу моделей. Тому час, зекономлений на обчисленнях, може значно перевищувати додаткову затримку, пов'язану з передачею запиту через мережу.

У проведеному експерименті хмарна модель відповіла за 1,23 с, а локальна — за 8,18 с.

4. Які ризики для приватності виникають при використанні хмарних API і що з цим роблять у комерційних проєктах?

При використанні хмарного API текст запиту передається на сервер стороннього провайдера. Тому в комерційних системах необхідно враховувати політику зберігання й обробки даних провайдера, обмежувати передачу конфіденційної інформації, контролювати доступ до API-ключів та за потреби анонімізувати дані.

Локальна модель у цьому аспекті має перевагу, оскільки обробка може виконуватися без передачі тексту на зовнішній сервер.

5. Що таке rate limit у безкоштовних тарифах і як розробники обходять це обмеження?

Rate limit — це встановлене провайдером обмеження на кількість запитів або токенів, які можна використати за певний проміжок часу. У реальних програмах такі обмеження враховують за допомогою черг запитів, повторних спроб із затримкою, кешування результатів та контролю частоти звернень до API.

Власні думки щодо локальних і хмарних моделей

На мою думку, локальну модель доцільно використовувати у випадках, коли важлива приватність даних, автономність та можливість працювати без підключення до Інтернету. Вона також дозволяє виконувати запити без залежності від зовнішнього API та його лімітів.

Хмарну модель я б обрав для завдань, де важливішими є якість, швидкість та можливість отримувати детальніші відповіді. У проведеному експерименті хмарна модель сформувала значно більшу відповідь і при цьому витратила лише 1,23 с, тоді як локальна — 8,18 с.

Недоліком хмарного підходу є залежність від Інтернету, API-ключа, лімітів провайдера та необхідність передавати текст запиту на зовнішній сервер. Тому вибір між локальною та хмарною моделлю залежить від конкретного завдання та вимог до приватності, швидкості й якості.

Висновок до Завдання 3

У ході виконання Завдання 3 було налаштовано єдиний програмний інтерфейс для роботи з локальною та хмарною мовними моделями. Для локальної роботи використовувалась llama3.2:3b через Ollama, а для хмарної — openai/gpt-oss-120b через API Groq.

Експеримент підтвердив, що завдяки OpenAI-сумісному API основна логіка програми залишається однаковою. Для перемикання між провайдерами достатньо змінити адресу сервера, API-ключ та назву моделі.

Локальна модель сформувала відповідь довжиною 283 символи за 8,18 с, використавши 92 completion-токени. Хмарна модель сформувала значно детальнішу відповідь довжиною 1285 символів за 1,23 с, використавши 482 completion-токени.

Отже, в проведеному експерименті хмарна модель забезпечила вищу швидкість та більш детальну відповідь. Водночас локальна модель має важливі переваги — автономність, відсутність залежності від зовнішнього API та можливість не передавати вхідні дані сторонньому серверу.

Крок 5. Вимірювання швидкості для різних провайдерів

На цьому етапі було досліджено швидкість роботи локальної та хмарної мовних моделей. Для оцінювання продуктивності використовувалися показники TTFT, загальний час формування відповіді та швидкість генерації в символах за секунду.

TTFT (Time to First Token) — час від моменту надсилання запиту до отримання першої частини відповіді. Цей показник особливо важливий для

інтерактивних систем, оскільки визначає, скільки часу користувач очікує перед початком появи тексту.

Окремо вимірювався загальний час відповіді та швидкість генерації. Для отримання першого токена використовувався потоковий режим `stream=True`.

Для забезпечення більш коректних результатів використовувався прогрів моделі, три повтори кожного вимірювання та медіана отриманих значень .

Завдання 4. Вимірювання швидкості: TTFT і залежність від довжини

запиту Мета завдання

Метою завдання було виміряти та порівняти швидкість локальної моделі `llama3.2:3b` та хмарної моделі `openai/gpt-oss-120b`, а також дослідити вплив типу запиту на TTFT і загальну швидкість формування відповіді.

Для варіанта №1 використовувалися три промпти.

Короткий промпт:

«Столиця

Франції?» Середній

промпт:

«Поясни в одному абзаці, що таке рекурсія.

» Довгий промпт:

«Напиши інструкцію з 10 пунктів, як почати вивчати

Python.» Крок 1. Створення програми `benchmark.py`

Для проведення вимірювань було створено файл `benchmark.py`.

```
import time
```

```
PROMPTS = {  
    "short": "Столиця Франції?",  
    "medium": "Поясни в одному абзаці, що таке рекурсія.",  
    "long": "Напиши інструкцію з 10 пунктів, як почати вивчати Python.",  
}
```

```

rows = []

for provider in ["local", "cloud"]:
    client, model = make_client(provider)

    measure(client, model, "pozirpiB")

    for name, prompt in PROMPTS.items():
        runs = [
            measure(client, model, prompt)
            for _ in range(3)
        ]

        rows.append({
            "provider": provider,
            "model": model,
            "prompt": name,
            "ttft_s": statistics.median(
                r["ttft_s"] for r in runs
            ),
            "total_s": statistics.median(
                r["total_s"] for r in runs
            ),
            "chars_per_s": statistics.median(
                r["chars_per_s"] for r in runs
            ),
        })

print(rows[-1])

```

```
with open("results.csv", "w", newline="", encoding="utf-8") as f:

    writer = csv.DictWriter(

        f,

        fieldnames=rows[0].keys()

    )

    writer.writeheader()

    writer.writerows(rows)
```

Для вимірювання часу використовувалася функція:

```
time.perf_counter()
```

Для отримання відповіді по частинах було встановлено:

```
stream=True
```

Момент отримання першої непорожньої частини відповіді фіксувався як TTFT.

Перед основними вимірюваннями виконувався прогрів:

```
measure(client, model, "розігрів")
```

Результат прогріву не враховувався. Кожна комбінація моделі та промпту запускалася тричі, після чого обчислювалася медіана.

Крок 2. Запуск benchmark

Програму було запущено командою:

```
python benchmark.py
```

Після завершення результати автоматично було збережено у файл:

```
results.csv
```

За результатами експерименту було отримано шість комбінацій: три типи промптів для локальної та три для хмарної моделі.

Рисунок 20 – Результати вимірювання швидкості локальної та хмарної моделей.

```
(venv) C:\Users\oleks\lab1>notepad benchmark.py

(venv) C:\Users\oleks\lab1>python benchmark.py
{'provider': 'local', 'model': 'llama3.2:3b', 'prompt': 'short', 'tftt_s': 2.142, 'total_s': 3.163, 'chars_per_s': 11.4}
{'provider': 'local', 'model': 'llama3.2:3b', 'prompt': 'medium', 'tftt_s': 2.173, 'total_s': 8.977, 'chars_per_s': 39.9}
{'provider': 'local', 'model': 'llama3.2:3b', 'prompt': 'long', 'tftt_s': 2.154, 'total_s': 66.357, 'chars_per_s': 32.6}
{'provider': 'cloud', 'model': 'openai/gpt-oss-120b', 'prompt': 'short', 'tftt_s': 0.395, 'total_s': 0.425, 'chars_per_s': 65.9}
{'provider': 'cloud', 'model': 'openai/gpt-oss-120b', 'prompt': 'medium', 'tftt_s': 0.312, 'total_s': 0.633, 'chars_per_s': 652.6}
{'provider': 'cloud', 'model': 'openai/gpt-oss-120b', 'prompt': 'long', 'tftt_s': 0.444, 'total_s': 2.265, 'chars_per_s': 1083.8}

Готово! Результати збережено у results.csv

(venv) C:\Users\oleks\lab1>ollama stop llama3.2:3b

(venv) C:\Users\oleks\lab1>python benchmark.py
{'provider': 'local', 'model': 'llama3.2:3b', 'prompt': 'short', 'tftt_s': 2.171, 'total_s': 2.81, 'chars_per_s': 8.3}
{'provider': 'local', 'model': 'llama3.2:3b', 'prompt': 'medium', 'tftt_s': 2.135, 'total_s': 8.879, 'chars_per_s': 42.0}
{'provider': 'local', 'model': 'llama3.2:3b', 'prompt': 'long', 'tftt_s': 2.185, 'total_s': 34.605, 'chars_per_s': 37.5}
{'provider': 'cloud', 'model': 'openai/gpt-oss-120b', 'prompt': 'short', 'tftt_s': 0.463, 'total_s': 0.49, 'chars_per_s': 57.2}
{'provider': 'cloud', 'model': 'openai/gpt-oss-120b', 'prompt': 'medium', 'tftt_s': 0.448, 'total_s': 0.706, 'chars_per_s': 559.5}
{'provider': 'cloud', 'model': 'openai/gpt-oss-120b', 'prompt': 'long', 'tftt_s': 0.344, 'total_s': 2.388, 'chars_per_s': 1209.6}

Готово! Результати збережено у results.csv
```

Крок 3. Холодний і теплий старт локальної моделі

Окремо було досліджено різницю між холодним і теплим стартом локальної моделі.

Перед холодним стартом модель було зупинено:

```
ollama stop llama3.2:3b
```

Для окремого вимірювання було створено програму `start_test.py`: `import time`
`from providers import make_client`

```
client, model = make_client("local")
```

```
prompt = "Столиця Франції?"
```

```
t_start = time.perf_counter()
```

```
first_token_at = None
```

```
parts = []
```

```
stream = client.chat.completions.create(
    model=model,
    messages=[
        {"role": "user", "content": prompt}
    ],
    stream=True,
)
```

```
for chunk in stream:
```

```
    piece = chunk.choices[0].delta.content
```

```
if piece:
    if first_token_at is None:
        first_token_at = time.perf_counter()

    parts.append(piece)

t_end = time.perf_counter()

print("Відповідь:", "".join(parts))
print(f"TTFT: {first_token_at - t_start:.3f} c")
print(f"Всього: {t_end - t_start:.3f} c")
```

Перший запуск після `ollama stop` використовувався як холодний старт. Другий запуск без зупинки моделі — як теплий старт.

Для холодного старту було отримано:

TTFT: 8.159 c

Всього: 9.024 c

Для теплого старту:

TTFT: 2.285 c

Всього: 5.785 c

Отже, TTFT холодного старту був приблизно у 3,6 раза більшим, ніж теплого. Це демонструє додаткові витрати часу на завантаження локальної моделі перед першим запитом.

Водночас відповіді в цих двох запусках мали різну довжину, тому загальний час не є настільки чистим показником ефекту холодного старту, як TTFT.

Рисунок 21 – Порівняння холодного та теплого старту локальної моделі llama3.2:3b.

```
(venv) C:\Users\oleks\lab1>ollama stop llama3.2:3b

(venv) C:\Users\oleks\lab1>python start_test.py
Відповідь: Столицею Франції є Paris.
TTFT: 8.159 c
Всього: 9.824 c

(venv) C:\Users\oleks\lab1>python start_test.py
Відповідь: Столиця Франції - Paris! Відомна своє красною архітектурою, культурним значенням і історичним скарбом цього міста. Paris розташований на півночі Франції, на rivière Seine.
TTFT: 2.285 c
Всього: 5.785 c
```

Крок 4. Результати вимірювань

Таблиця 7 – Результати вимірювання швидкості

Розміщення Модель		Промпт	TTFT, с	Всього, с	Симв./с
локальна	llama3.2:3b	короткий	2,142	3,163	11,4
локальна	llama3.2:3b	середній	2,173	8,977	39,9
локальна	llama3.2:3b	довгий	2,154	66,357	32,6
хмарна	openai/gpt-oss-120b	короткий	0,395	0,425	65,9
хмарна	openai/gpt-oss-120b	середній	0,312	0,633	652,6
хмарна	openai/gpt-oss-120b	довгий	0,444	2,265	1083,8

За отриманими результатами хмарна модель мала менший TTFT для всіх трьох типів промтів. Для локальної моделі TTFT залишався приблизно на рівні 2,1–2,2 c, а для хмарної — приблизно 0,3–0,4 c.

Особливо значною була різниця для довгого промпту: локальна модель завершила відповідь за 66,357 с, тоді як хмарна — за 2,265 с.

Таблиця 8 – Холодний і теплий старт локальної моделі

Показник	Холодний старт	Теплий старт
----------	----------------	--------------

TTFT, с	8,159	2,285
---------	-------	-------

Всього, с	9,024	5,785
-----------	-------	-------

Холодний запуск мав значно більший TTFT через необхідність завантаження моделі перед початком генерації. Після завантаження моделі наступний запит почав оброблятися значно швидше.

Крок 5. Побудова стовпчикової діаграми

Для наочного порівняння швидкості генерації було побудовано стовпчикову діаграму за допомогою бібліотеки `matplotlib`.

Для її встановлення було виконано:

```
pip install matplotlib
```

Після цього створено файл `chart.py`:

```
import matplotlib.pyplot as plt
```

```
labels = [  
    "Локальна\пкороткий",  
    "Локальна\псередній",  
    "Локальна\пдовгий",  
    "Хмарна\пкороткий",  
    "Хмарна\псередній",  
    "Хмарна\пдовгий"  
]
```

```
speed = [  
    11.4,  
    39.9,  
    32.6,  
    65.9,  
    652.6,  
    1083.8  
]
```

```
]

plt.figure(figsize=(11, 6))

bars = plt.bar(labels, speed)

plt.title("Швидкість генерації локальної та хмарної моделей")
plt.xlabel("Модель та тип промπτу")
plt.ylabel("Швидкість, символів/с")

for bar, value in zip(bars, speed):
    plt.text(
        bar.get_x() + bar.get_width() / 2,
        bar.get_height() + 10,
        str(value),
        ha="center"
    )

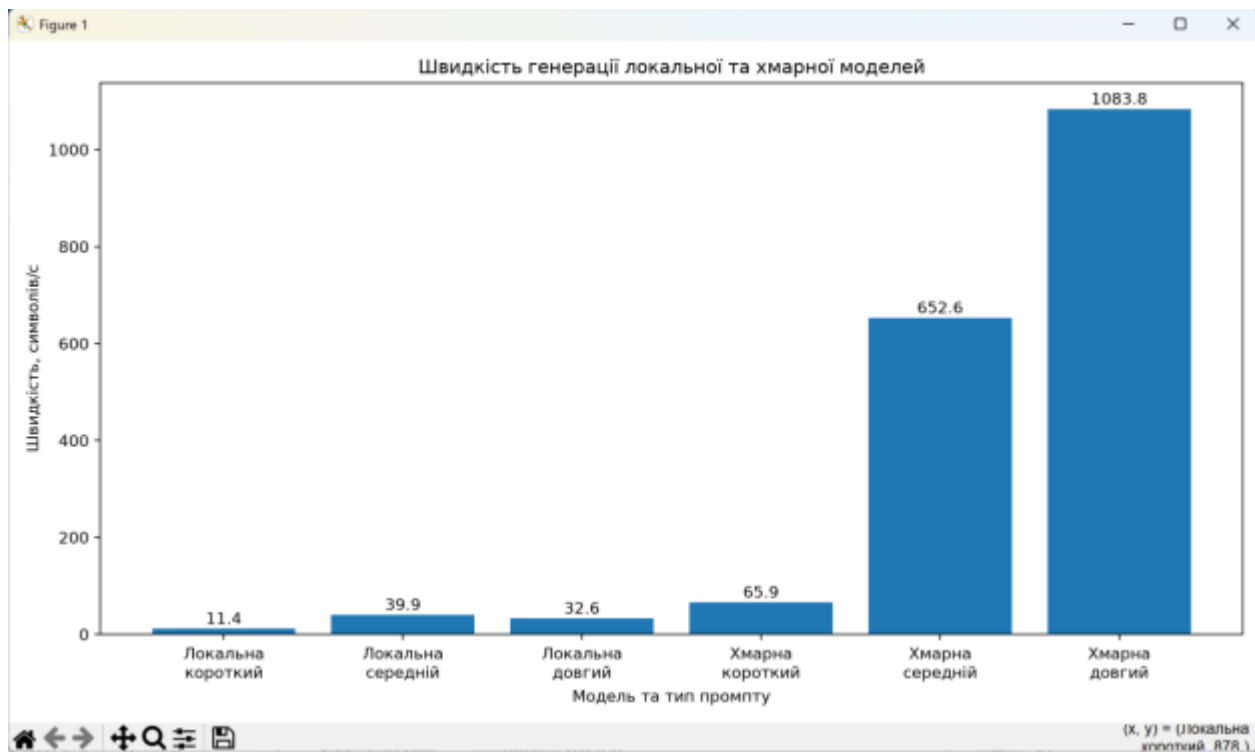
plt.tight_layout()
plt.savefig("speed_chart.png", dpi=300)
plt.show()
```

Програму було запущено:

`python chart.py`

За результатами було створено файл `speed_chart.png`.

Рисунок 22 – Швидкість генерації локальної та хмарної моделей для промптів різної довжини.



Діаграма наочно показує значну перевагу хмарної моделі за виміряною швидкістю в символах за секунду. Найвище значення було отримано для хмарної моделі з довгим промптом — 1083,8 симв./с.

Крок 6. Відповіді на питання

1. У якій моделі кращий TTFT і чому саме в неї?

У проведеному експерименті кращий TTFT мала хмарна модель: 0,312–0,444 с проти приблизно 2,142–2,173 с у локальній. Попри передачу даних через Інтернет, серверна інфраструктура хмарного провайдера забезпечила швидшу обробку запиту.

2. Чи збігається «найшвидша за TTFT» з «найшвидшою за символами/с»? Якщо ні — поясніть.

Так, у проведеному експерименті хмарна модель була швидшою за обома показниками. Вона мала менший TTFT і водночас досягла значно більшої швидкості генерації — до 1083,8 симв./с.

3. Як змінилося співвідношення локальної та хмарної моделі при переході від короткого промпту до довгого?

На короткому промпті загальний час становив 3,163 с для локальної та 0,425 с для хмарної моделі. На довгому промпті різниця стала набагато помітнішою: 66,357 с проти лише 2,265 с, тому перевага хмарної моделі збільшилася зі зростанням обсягу генерації.

4. Наскільки різняться три повтори одного виміру? Чи достатньо було б одного запуску?

У підсумковій таблиці використано медіану трьох запусків, тому окремі значення повторів у ній не збережені. Одного запуску недостатньо для надійного вимірювання, оскільки на час можуть впливати поточне навантаження комп'ютера, мережеві затримки та випадкові коливання генерації.

5. Чому холодний старт повільніший за теплий?

При холодному старті локальна модель ще не завантажена для виконання запиту, тому потрібен додатковий час на її підготовку. Після першого запуску модель залишається завантаженою, тому TTFT зменшився з 8,159 с до 2,285 с.

Крок 7. Питання до чат-бота

1. Що фізично відбувається під час TTFT і чому цей показник вимірюють окремо від загальної швидкості генерації?

До появи першого токена система повинна прийняти запит, підготувати модель та обробити вхідний контекст. TTFT вимірюється окремо, оскільки він характеризує початкову затримку, яку безпосередньо відчуває користувач до початку появи відповіді.

2. Чому хмарна модель може мати гірший TTFT, але кращу швидкість генерації, ніж локальна?

Теоретично мережеве з'єднання може збільшувати TTFT хмарної моделі, однак після початку генерації потужна серверна інфраструктура здатна формувати відповідь швидше за локальний комп'ютер. У нашому

експерименті хмарна модель виявилася швидшою навіть за TTFT, тому мережеві витрати не перекрыли перевагу серверної інфраструктури.

3. Чому перший запуск локальної моделі повільніший за наступні і що таке холодний старт?

Холодним стартом називається перший запуск, коли модель ще необхідно завантажити та підготувати до роботи. У нашому експерименті холодний TTFT становив 8,159 с, а теплий — 2,285 с, що демонструє значний вплив початкового завантаження.

4. Чому при вимірюванні продуктивності беруть медіану кількох запусків, а не одне значення?

Окремий вимір може бути спотворений випадковою затримкою мережі, навантаженням процесора або іншими фоновими процесами. Медіана кількох повторів менше залежить від одиничних аномально великих або малих значень і тому краще характеризує типову продуктивність.

5. Чому обробка вхідного запиту йде швидше, ніж генерація відповіді такої ж довжини?

Вхідний текст модель може обробляти паралельно великими блоками. Генерація відповіді переважно відбувається послідовно: наступний токен залежить від уже сформованих токенів, тому процес генерації має інші обмеження продуктивності.

Власні думки щодо показників швидкості

На мою думку, для інтерактивного чат-бота одним із найважливіших показників є TTFT, оскільки саме він визначає, скільки часу користувач бачить порожній екран до початку відповіді. Навіть якщо повна відповідь генерується довго, рання поява першого тексту створює відчуття швидкої реакції системи.

Для фонові обробки великих обсягів тексту важливішою є загальна пропускна здатність — швидкість генерації та повний час виконання. У такому випадку користувач не спостерігає за появою кожного токена, тому важливіше, скільки роботи система здатна виконати за певний проміжок часу.

Проведений експеримент також показав, що не можна оцінювати продуктивність мовної моделі лише одним показником. TTFT характеризує початкову затримку, тоді як загальний час і швидкість у символах за секунду характеризують подальшу генерацію.

Висновок до Завдання 4

У ході виконання Завдання 4 було досліджено продуктивність локальної моделі llama3.2:3b та хмарної openai/gpt-oss-120b. Для кожної моделі було виконано вимірювання на короткому, середньому та довгому промптах із використанням потокової генерації.

Результати показали, що в проведеному експерименті хмарна модель мала менший TTFT для всіх трьох промптів. Для неї значення становили 0,395 с, 0,312 с та 0,444 с, тоді як для локальної — 2,142 с, 2,173 с та 2,154 с відповідно.

Особливо значна різниця спостерігалася при довгій генерації. Локальна модель витратила 66,357 с, тоді як хмарна — лише 2,265 с. Максимальна виміряна швидкість хмарної моделі становила 1083,8 симв./с, а локальної — 39,9 симв./с.

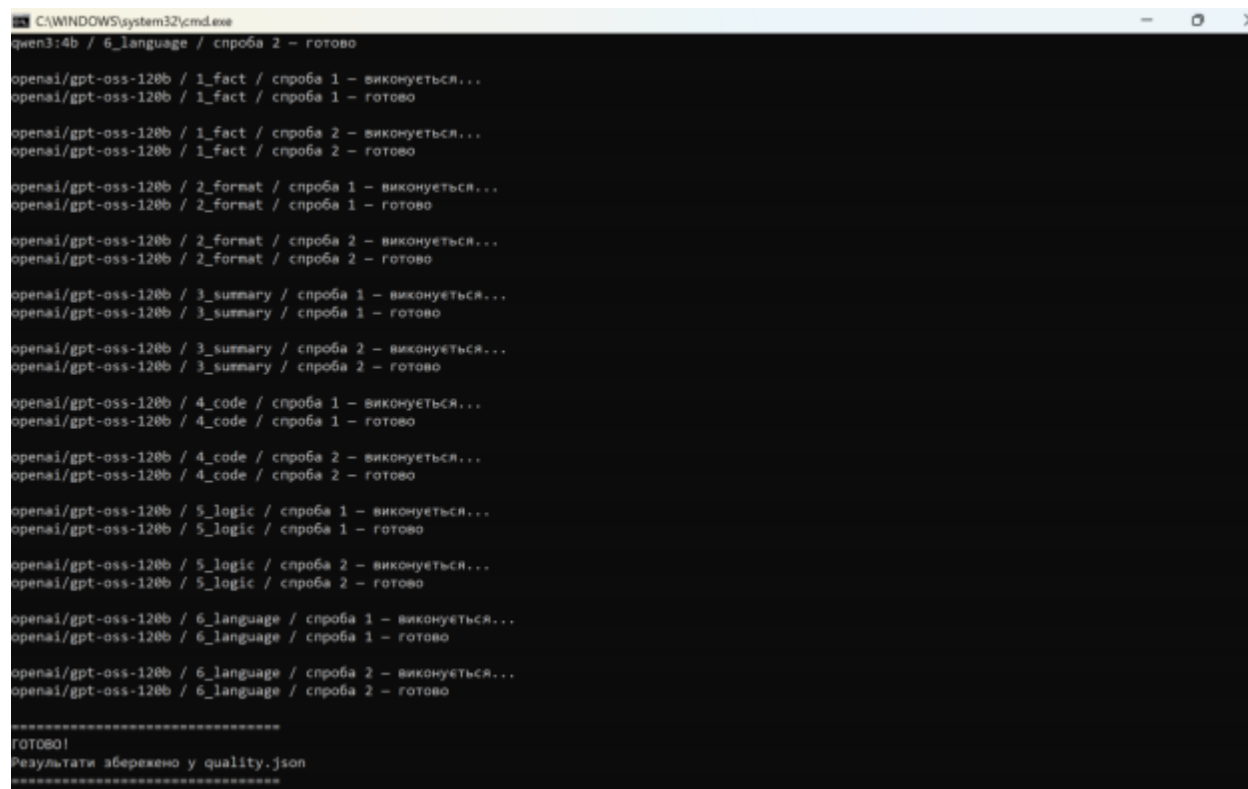
Окреме дослідження холодного та теплого старту показало, що TTFT локальної моделі зменшився з 8,159 с до 2,285 с після її завантаження. Отже, стан моделі в пам'яті має суттєвий вплив на початкову затримку.

За результатами роботи можна зробити висновок, що для інтерактивних застосунків важливо враховувати TTFT, а для обробки великих обсягів даних — загальну швидкість генерації та час виконання. У проведених вимірюваннях хмарна модель продемонструвала значну перевагу за обома типами показників.

Крок 6. Оцінювання якості моделей за рубрикою

Для оцінювання якості було використано три мовні моделі: локальні llama3.2:3b і qwen3:4b та хмарну openai/gpt-oss-120b. Кожна модель виконувала шість завдань, причому кожне завдання запускалося двічі, оскільки відповіді мовних моделей можуть відрізнятися між повторними запусками.

Оцінювання проводилося за шкалою від 0 до 2 балів відповідно до критеріїв методичних вказівок. Максимальна оцінка для однієї моделі становить 12 балів.



```
C:\WINDOWS\system32\cmd.exe
qwen3:4b / 6_language / спроба 2 – готово

openai/gpt-oss-120b / 1_fact / спроба 1 – виконується...
openai/gpt-oss-120b / 1_fact / спроба 1 – готово
openai/gpt-oss-120b / 1_fact / спроба 2 – виконується...
openai/gpt-oss-120b / 1_fact / спроба 2 – готово

openai/gpt-oss-120b / 2_format / спроба 1 – виконується...
openai/gpt-oss-120b / 2_format / спроба 1 – готово
openai/gpt-oss-120b / 2_format / спроба 2 – виконується...
openai/gpt-oss-120b / 2_format / спроба 2 – готово

openai/gpt-oss-120b / 3_summary / спроба 1 – виконується...
openai/gpt-oss-120b / 3_summary / спроба 1 – готово
openai/gpt-oss-120b / 3_summary / спроба 2 – виконується...
openai/gpt-oss-120b / 3_summary / спроба 2 – готово

openai/gpt-oss-120b / 4_code / спроба 1 – виконується...
openai/gpt-oss-120b / 4_code / спроба 1 – готово
openai/gpt-oss-120b / 4_code / спроба 2 – виконується...
openai/gpt-oss-120b / 4_code / спроба 2 – готово

openai/gpt-oss-120b / 5_logic / спроба 1 – виконується...
openai/gpt-oss-120b / 5_logic / спроба 1 – готово
openai/gpt-oss-120b / 5_logic / спроба 2 – виконується...
openai/gpt-oss-120b / 5_logic / спроба 2 – готово

openai/gpt-oss-120b / 6_language / спроба 1 – виконується...
openai/gpt-oss-120b / 6_language / спроба 1 – готово
openai/gpt-oss-120b / 6_language / спроба 2 – виконується...
openai/gpt-oss-120b / 6_language / спроба 2 – готово

Готово!
Результати збережено у quality.json
```

Рисунок 23 – Виконання тестів якості мовних моделей

Для завдання 1 необхідно було встановити правильний рік заснування Одеського національного університету імені І. І. Мечникова. Еталонне значення — 1865 рік.

Отримані відповіді:

- llama3.2:3b — 1855 та 1897;
- qwen3:4b — 1920 та 1913;
- openai/gpt-oss-120b — 1918 та 1865.

Отже, локальні моделі в обох прогонах назвали неправильні роки. Хмарна модель у першому прогоні також помилилася, але в другому вказала правильний рік. За критеріями методички випадковий правильний результат при розбіжності прогонів не можна вважати стабільним знанням факту.

Pasted markdown

Для завдання 2 було перевірено дотримання формату JSON. Згідно з методикою, відповідь повинна без змін оброблятися функцією `json.loads()`: у такому випадку виставляється 2 бали . Pasted markdown

llama3.2:3b та openai/gpt-oss-120b повертали синтаксично коректний JSON без Markdown-обгортки. У qwen3:4b у проведеному запуску відповіді для цього завдання виявилися порожніми .

Для завдання 3 моделі повинні були стиснути наданий текст до двох речень і зберегти всі числові дані. llama3.2:3b втратила частину чисел. Після повного повторного запуску openai/gpt-oss-120b зберегла числа 21, 1944, 1945, 1965, 1978, 1980, 9, 2020 та 155 і вклалася у два речення. Для qwen3:4b відповіді залишилися порожніми .

Для завдання 4 потрібно було реалізувати функцію `is_palindrome(s)`, яка ігнорує пробіли та регістр . llama3.2:3b і openai/gpt-oss-120b сформували працездатні реалізації, що виконують поставлену умову .

Наприклад, llama3.2:3b згенерувала:

```
def is_palindrome(s):    s = ''.join(c for c in s if c.isalnum()).lower()    return s == s[::-1]
```

Для завдання 5 правильна відповідь визначається за принципом включення - виключення:

$$12 + 9 - 4 = 17$$
$$25 - 17 = 8$$

Отже, 8 студентів не знають жодної з двох мов .

llama3.2:3b у першому прогоні використала потрібну формулу, але припустилася арифметичної помилки й отримала 4. qwen3:4b у повній відповіді правильно отримала 8, однак інший прогін був обірваний. openai/gpt-oss-120b в обох прогонах правильно показала хід розв'язання та отримала 8.

Для завдання 6 оцінювалася якість української мови під час пояснення різниці між стеком і чергою. llama3.2:3b сформувала некоректний текст із беззмістовними твердженнями та мовними помилками. openai/gpt-oss-120b правильно пояснила принципи LIFO та FIFO природною українською мовою

у двох абзацах. У qwen3:4b відповіді в проведеній конфігурації були порожніми.

```
{

  "llama3.2:3b":

  { "1_fact": [

    "1855",

    "1897."

  ],

  "2_format": [

    "{ \"name\": \"Одеський національний університет імені І.І .  
Мечникова\", \"year\": 1940, \"city\": \"Одеська\" }",

    "{ \"name\": \"Одеський національний університет імені І . І .  
Мечникова\",  
\"year\": 1948, \"city\":  
\"Одесса\" }" ],

  "3_summary": [

    "Наступний текст стиснутий до двох речей, збережена інформація про  
числа:\n\nОдеський державний університет відновив свою роботу в Одесі 21  
квітня 1944 року . Університетом було присвоєно ім'я І . І . Мечникова в  
1945 році, а у 1965 році він був нагороджений орденом Трудового Червоного  
Папора.",

    "Наступний текст зменшується до дві речення з збереженням усіх  
чисел:\n\nОдеський державний університет відновив свою роботу в Одесі 21  
квітня 1944 року . У 1945 році йому було присвоєно ім'я І . І .  
Мечникова ." ],

  "4_code": [
```

```
"""python\ndef is_palindrome(s):\n    s = ''.join(c for c in s if\n    c.isalnum()).lower()\n    return s == s[::-1]\n""",
```

```
"""python\ndef is_palindrome(s):\n    s = ''.join(c for c in s if\n    c.isalnum()).lower()\n    return s == s[::-1]
```

```
1]\n"""] ,
```

```
"5_logic": [
```

"Давайте посмуємо хід розв'язання:\n\n1. Обігняйте статистику за кожною мовою:\n - Студентів, які знають Python: 12\n - Студентів, які знають Java: 9\n - Студентів, які знають обидві мови: 4\n2. Враховуйте студентів, яких не

знають жодної з цих мов . Для цього нам потрібно знати, скільки студентів не знають жодної з двох мов, а саме ті, які не знають жодної з них .\n3. використовуйте формулу для обчислення кількості студентів, які не знають жодної з двох мов:\n\n Кількість студентів, які не знають жодної з двох мов = Кількість студентів в групі - Кількість студентів, які знають Python -

Кількість студентів, які знають Java + Кількість студентів, які знають обидві мови\n\n4. Рассчитайте частоту:\n\n Кількість студентів, які не знають жодної з двох мов = 25 - 12 - 9 + 4\n= 25 - 21\n= 4\n\n5. Погледьте в результаті!",

"Височна quality вашого запиту!\n\nДавайте розв'яжемо проблему step by step:\n\n1. Кількість студентів, які знають Python: 12\n2. Кількість студентів,

які знають Java: 9\n3. Кількість студентів, які знають обидві мови: 4\n\nЧіємо, що студентів, які знають обидві мови, уже включені у числа студентів, які знають Python та Java. Meaning, якщо ми відрахуємо вихідних студентів, які знають обидві мови, то їх не потрібно відраховувати

двічі.\n\nДоволена! \n\nТепер, нас потрібно знати, скільки студентів не знають жодної з цих мов . Для цього ми повинні знати, скільки студентів не знають жодної з двох мов .\n\nМи знаємо, що в загальній кількості 25 студентів . Знаємо, що 12 студентів знають Python, а 9 студентів знають Java. Пізніше ми знаємо, що 4 студента знають обидві мови .

\n\nВисококваліфікований крок!\n\nЧіємо, що студентів, які знають обидві мови, не повинні бути враховані двічі . Пізніше врахуємо, що 4 студента вже включені у числа студентів, які знають Python та Java.\n\nТепер, нас потрібно знати, скільки студентів не знають жодної з двох мов . Для цього ми повинні знати, скільки студентів не знають жодної з двох мов .\n\nМи знаємо, że в

загальній кількості 25 студентів . Знаємо, що 12 студентів знають Python, а 9 студентів знають Java. Пізніше ми знаємо, що 4 студента знають обидві

мови.\n\nАналогічно, не можна рахувати студентів, які не знають жодної з двох мов, двічі . Пізніше ми рахуємо студентів , які не знають жодної з двох мов, лише один раз.\n\nТепер, нас необхідно знати, скільки студентів не знають жодної з двох мов . Для цієї мети нам потрібно знати, скільки студентів не знають жодної

з"],

"6_language": [

"Чим відрізняється від черги:\n\n* Стек не має сітки, а черга має сітку з дюжинами сіток з різної величини і форми, що допомагає в'язанню гусеніна лебедя.\n* Стек виготовляється з гусеніна лебедя, а черга виготовляється з інших видів чумазової тканини."

"Стек відрізняється від черги наступними особливостями:\n\n* Стек складається з тонких, проникливих молекул, які можуть проникати в різні речовини та атоми, створюючи особливу взаємодію . \n* ЧЕРГ є повністю нереєстрованою молекулою з дуже великотеносністю, яку виробить навіть на більшій кількості ядерних реакцій ."

]

},

"qwen3:4b": {

"1_fact": [

"" ,

"" ""

],

"2_format":

["" ,

"" ""

],

```
"3_summary": [
  "",
  ""

],

"4_code":
[ "",
  ""

],

"5_logic":
[ "",
  "Щоб визначит"
],

"6_language": [
  "",
  ""

]

},

"openai/gpt-oss-120b":
{ "1_fact": [
  "1918",
  "1865"
```

```

],

"2_format": [

    {"\name\":"Одеський національний університет імені І .
І . Мечникова\","\year\:1918,\city\":"Одеса\""},

    {"\name\":"Одеський національний університет імені І . І .
Мечникова\","\year\:1865,\city\":"Одеса\

"}" ],

"3_summary": [

    "Одеський державний університет відновив роботу в
Одесі 21 квітня 1944 року, у 1945 отримав назву І. І. Мечникова, у 1965
був нагороджений орденом Трудового Червоного Прапора, а у 1978 включений
до переліку провідних університетів СРСР. До кінця 1980-х років він
складався з 9 фак",

    "Одеський державний університет відновив свою роботу в
Одесі 21 квітня 1944 року, у 1945 році отримав ім'я І. І. Мечникова,
у 1965 році був нагороджений орденом Трудового Червоного Прапора та у
1978 році включений до переліку провідних університетів СРСР . До
кінця 1980-х років університет складався з 9 факультетів, а у 2020

ро" ],

"4_code": [

    "`python\ndef is_palindrome(s):\n    cleaned = ''.join(ch.lower()
for ch in s if ch != ' ')\n    return cleaned == cleaned[::-1]\n`",

    "`python\ndef is_palindrome(s: str) -> bool:\n    \\\\\"Return True
if `s` is a palindrome, ignoring spaces and case.\\\\\n    cleaned =
''.join(ch.lower() for ch in s if not ch.isspace())\n    return cleaned
== cleaned[::-1]\n`"

],

"5_logic": [

```

***Розв'язок за допомогою принципу включення-виключення**\n\nНехай
 $|P|$ - кількість студентів, які знають Python; $|J|$ -
 кількість студентів, які знають Java; $|P \cap J|$ - кількість студентів,
 які знають обидві мови; $|U|$ - загальна кількість студентів у
 групі.\n\nЗ умов задачі маємо $|P| = 12, |J| = 9, |P \cap J| = 4, |U| = 25$.\n\n---\n\n### Крок 1. Знайти кількість студентів, які знають
 принаймні одну мову\n\nЗа принципом включення-виключення:
 $|P \cup J| = |P| + |J| - |P \cap J|$
 Підставляємо дані:
 $|P \cup J| = 12 + 9 - 4 = 17$.\n\nОтже, 17 студентів знають Python **або** Java (можуть знати і те, і інше).\n\n---\n\n### Крок 2. Знайти кількість студентів, які **не знають жодної** мови
 $|U \setminus (P \cup J)| = |U| - |P \cup J|$
 $|U \setminus (P \cup J)| = 25 - 17 = 8$.\n\n---\n\n## Відповідь\n\n**8 студентів** у групі не знають жодної з цих мов.\n\n---\n\n### Пояснювальна діаграма (за бажанням)\n\n


***Розв'язок за принципом включення-виключення**\n\nНехай $|P| = 12$; $|J| = 9$;
 $|P \cap J| = 4$ -
 студенти, які знають обидві мови.\n\n1. **Знаходимо кількість студентів, які знають хоча б одну мову**
 За принципом включення-виключення
 $|P \cup J| = |P| + |J| - |P \cap J| = 12 + 9 - 4 = 17$.\n\nОтже, 17 студентів знають Python **або** Java (можливо, обидві).\n\n2. **Знаходимо кількість студентів, які не знають жодної з мов**
 У групі всього 25 студентів, тому $|U \setminus (P \cup J)| = 25 - |P \cup J| = 25 - 17 = 8$.\n\n---\n\n### Відповідь\n\n**8 студентів** у групі не знають жодної з мов Python і Java."],
 "6_language": [

***Стек** - це структура даних типу «останній прийшов, перший вийшов» (LIFO). При додаванні елементу він розташовується на верхній позиції, а видаляти можна лише цей верхній елемент. Уявіть собі стопку книг: нова книга кладеться зверху, а зняти можна лише ту, що зараз знаходиться на верху. Операції `push` (додати) і `pop` (видалити) працюють за постійний час $O(1)$ і не вимагають перегляду інших елементів.\n\n**Черга** - це структура даних типу «перший прийшов,

перший вийшов» (FIFO). Елемент додається в кінець (в хвостову частину), а видаляється з початку (з голови). Це схоже на чергу в банку: новий клієнт стає в кінець, а обслуговують тих, хто стоїть попереду. Операції `enqueue` (дати) і `dequeue` (видалити) також виконуються за $O(1)$, проте порядок доступу до елементів протилежний стеку.",

"Стек**** – це структура даних типу «останній прийшов, перший пішов» (LIFO – **last-in, first-out**). Новий елемент додається лише на верхню позицію (операція **push**), а видаляти можна лише з тієї ж вершини (операція **pop**). Така організація схожа на стопку книг: щоб дістати книгу, яку поклали останньою, потрібно зняти всі інші зверху. Стек часто використовується для збереження контексту викликів функцій, реалізації зворотного обходу графів, а також у алгоритмах, де важливий швидкий доступ лише до останнього елемента.\n\n****Черга**** працює за принципом «перший прийшов, перший пішов» (FIFO – **first-in, first-out**). Елемент додається в кінець черги (операція **enqueue**), а видаляється – з її початку (операція **dequeue**). Уявіть собі чергу в банку: нові клієнти стають в кінець, а обслуговуються ті, хто стоїть попереду. Черги застосовують у моделях обробки завдань, у мережевих буферах, у плануванні процесів та в алгоритмах пошуку в ширину, де важливо обробляти елементи у порядку їх надходження."

```

    ]
}
}

```

Результати двох прогонів завдань оцінювання

Таблиця 13 – Оцінка якості за рубрикою

Завдання	Плата3.2:3b	qwen3:4b	Хмарна модель
1. Факт	0	0	1
2. Формат	2	0	2
3. Стиснення	0	0	2
4. Код	2	0	2

Завдання	llama3.2:3b	qwen3:4b	Хмарна модель
5. Логіка	0	1	2
6. Українська	0	0	2
Разом /12	4/12	1/12	11/12

Примітка. Під час роботи qwen3:4b через використану локальну конфігурацію значна частина відповідей поверталася порожньою або обірваною. Тому отриманий бал характеризує результат саме проведеного експерименту, а не загальну якість моделі в усіх можливих конфігураціях.

Таблиця 14 – Розбіжність оцінок між двома прогонами

Завдання	llama3.2:3b	qwen3:4b	Хмарна модель
1. Факт	Ні	Ні	Так
2. Формат	Ні	Ні	Ні
3. Стиснення	Ні	Ні	Ні
4. Код	Ні	Ні	Ні
5. Логіка	Ні	Так	Ні
6. Українська	Ні	Ні	Ні

Розбіжність тут визначалась за оцінкою результату, а не просто за різницею формулювань відповідей.

Крок 6. Відповіді на питання

1. Яка модель набрала найбільше балів і на яких саме завданнях виникла основна різниця?

Найбільше балів у проведеному тестуванні набрала openai/gpt-oss-120b — 11 із 12. Основна різниця проявилася у стислому викладі, логічному міркуванні та якості української мови.

2. На яких завданнях оцінки двох прогонів розійшлися? Що це говорить про надійність моделі?

Помітна розбіжність була у фактичному завданні хмарної моделі та логічному завданні qwen3:4b. Це показує, що результат одного запуску не завжди є достатнім для оцінювання надійності моделі.

3. Чи виконала хоч одна модель вимогу «без markdown» у завданні 2?

Так. llama3.2:3b та openai/gpt-oss-120b повернули JSON без Markdown-обгортки, який можна було передати у `json.loads()` без попереднього видалення ````json`.

4. Чи були проблеми з українською мовою — русизми, кальки, перехід на іншу мову?

Так. Особливо помітними вони були у llama3.2:3b: відповідь містила неприродні конструкції та фактично втратила зміст питання. Хмарна модель у цьому тесті використовувала природнішу українську мову та правильну термінологію.

Крок 7. Приклад провалу моделі

Найбільш показовим виявився результат llama3.2:3b у завданні на якість української мови.

Повний текст запиту:

Чим стек відрізняється від черги? Поясни українською, два абзаци.

Повний текст відповіді моделі:

Чим відрізняється від черги:

- Стек не має сітки, а черга має сітку з дюжинами сіток з різної величини і форми, що допомагає в'язанню гусеніна лебедя.
- Стек виготовляється з гусеніна лебедя, а черга виготовляється з інших видів чумазової тканини.

Правильна відповідь: стек — це структура даних, що працює за принципом LIFO (Last In, First Out), тобто останній доданий елемент видаляється першим. Черга працює за принципом FIFO (First In, First Out), тобто першим видаляється елемент, який був доданий першим.

Модель неправильно інтерпретувала поняття «стек» і «черга» та згенерувала інформацію, яка не має відношення до структур даних. Такий результат є прикладом галюцинації: текст має зовнішню мовну форму відповіді, але його зміст є неправильним і частково беззмістовним.

```
"6_language": [{"text": "Чим відрізняється від черги:\n\n* Стек не має сітки, а черга має сітку з дежинами сіток з різної величини і форми, що допомагає в'язанню гусеніна лебедя.\n* Стек виготовляється з гусеніна лебедя, а черга виготовляється з інших видів чумазової тканини."}, {"text": "Стек відрізняється від черги наступними особливостями:\n\n* Стек складається з тонких, проникливих молекул, які можуть проникати в різні речовини та атоми, створюючи особливу взаємодію.\n* ЧЕРГ є повністю нересторованою молекулою з дуже великотеносністю, яку виробить навіть на більшій кількості ядерних реакцій."}]
```

Рисунок 24 – Приклад невдалої відповіді локальної моделі

Крок 8. Питання до чат-бота

1. Я оцінював мовні моделі за рубрикою з шести завдань і трибальною шкалою. Які ще підходи до оцінювання якості мовних моделей використовують на практиці?

На практиці застосовують автоматичні benchmark-тести, експертне оцінювання людьми, попарне порівняння відповідей, unit-тести для програмного коду та перевірку за еталонними відповідями. Для оцінювання стабільності один набір тестів також запускають багаторазово.

2. Чому мовні моделі часто ігнорують пряму інструкцію не використовувати markdown при генерації JSON?

Модель генерує текст на основі засвоєних шаблонів, а в навчальних даних JSON і код дуже часто оформлені Markdown-блоками. Через це такий шаблон іноді переважає навіть над прямою інструкцією користувача.

3. Чому малі мовні моделі гірше пишуть українською, ніж англійською?

Однією з причин є менша представленість якісних україномовних текстів у навчальних даних порівняно з англійськими. Через меншу кількість параметрів невеликим моделям також складніше стабільно відтворювати граматику, термінологію та контекст менш представлених мов.

4. Чому та сама модель на однаковий запит може дати відповіді різної якості при повторному запуску?

Генерація мовної моделі може бути стохастичною: на кожному кроці існує декілька можливих наступних токенів. Тому різні запуски одного запиту можуть привести до різних ланцюжків генерації та різної якості кінцевої відповіді.

5. Що таке галюцинація мовної моделі і чому модель радше вигидає факт, ніж визнає незнання?

Галюцинація — це генерація неправильної або вигаданої інформації, поданої як достовірної. Мовна модель передусім прогнозує правдоподібне продовження тексту, тому без додаткової перевірки джерел вона може сформулювати переконливу, але фактично неправильну відповідь.

Власні думки

Оцінювання за рубрикою виявилось достатньо зручним, оскільки для всіх моделей використовувалися однакові завдання та однакові критерії.

Водночас результати qwen3:4b показали, що під час експерименту потрібно враховувати не лише можливості самої моделі, а й технічні особливості її запуску.

Найпоказовішими виявилися завдання на фактичні знання, логічне міркування та українську мову. Фактичне завдання продемонструвало можливість галюцинацій, а повторні запуски показали, що модель може давати різні відповіді на однаковий запит.

Для роботи з українськомовним текстом серед протестованих конфігурацій я б обрав openai/gpt-oss-120b, оскільки саме вона в проведених тестах продемонструвала найбільш природну українську мову та правильне використання термінології.

Висновок

У результаті оцінювання за рубрикою llama3.2:3b отримала 4/12, qwen3:4b — 1/12, а openai/gpt-oss-120b — 11/12. Отримані результати показали, що швидкість моделі не є достатнім показником її практичної придатності:

окремо необхідно перевіряти фактичну точність, формат, роботу з наданим текстом, код, логічне міркування та якість української мови .

Повторні прогони також виявили нестабільність деяких відповідей . Тому для практичного використання LLM важливо не покладатися на одиничний результат, а застосовувати перевірки, структурований формат і, за потреби, зовнішні джерела інформації.

Крок 7. Дослідження впливу temperature і top_p

На цьому етапі було досліджено вплив параметрів temperature та top_p на випадковість і різноманітність відповідей мовної моделі.

Параметр temperature впливає на розподіл імовірностей наступного токена . При низькому значенні модель частіше обирає найбільш імовірні варіанти, тому відповіді стають передбачуванішими . При збільшенні temperature зростає ймовірність вибору менш очікуваних токенів, що збільшує різноманітність результатів.

Параметр top_p працює інакше: він обмежує множину токенів, які можуть брати участь у виборі, залишаючи найбільш імовірні варіанти до досягнення заданої сумарної ймовірності.

Завдання 6. Керування випадковістю: temperature і top_p

Для виконання завдання було використано варіант №1.

Творче завдання:

Запропонувати назву стартапу в галузі логістики.

Фактичне питання:

У якому році засновано Національний університет «Одеська політехніка»?

Для перевірки фактичної відповіді було використано офіційний сайт Національного університету «Одеська політехніка» . В офіційній довідці зазначено, що університет засновано у 1918 році.Odesa Polytechnic University

Офіційна довідка Національного університету «Одеська політехніка»

1. Дослідження впливу temperature

Для творчого завдання було виконано по п'ять запусків при значеннях:

temperature = 0.0

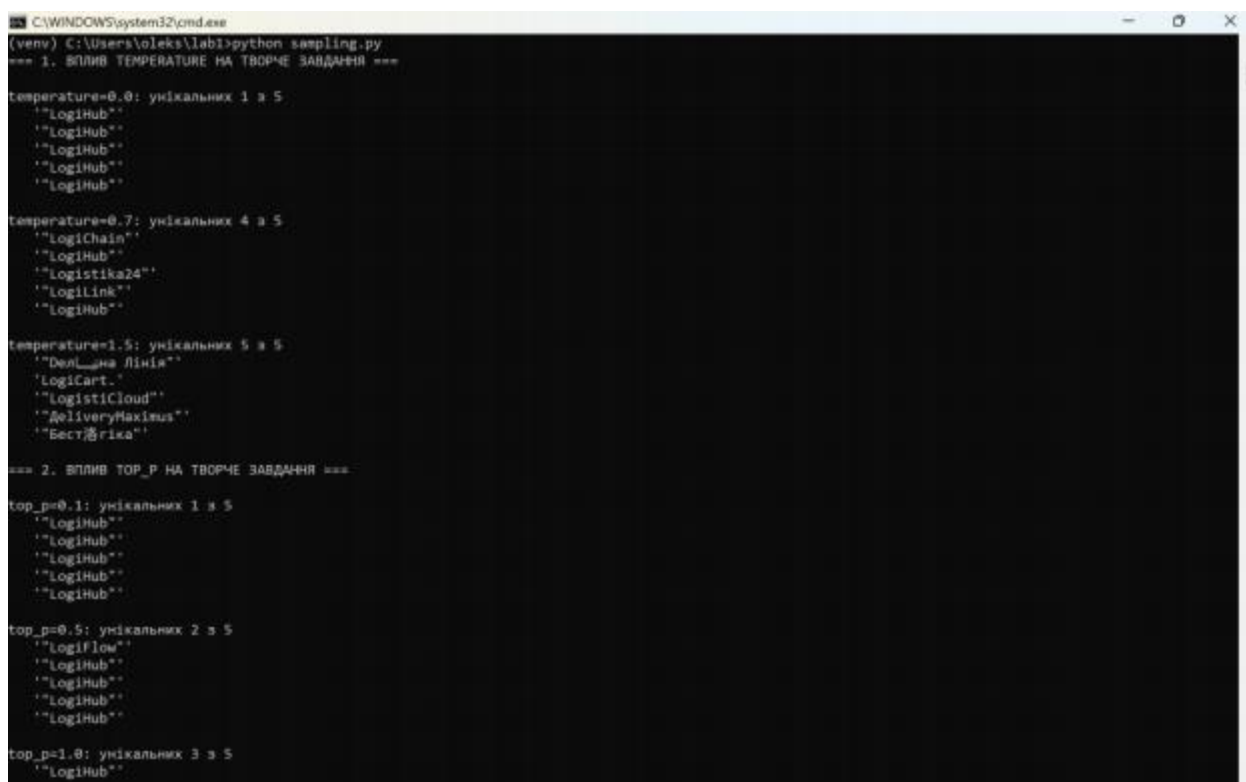
temperature = 0.7

temperature = 1.5

При temperature=0.0 усі п'ять запусків дали однакову назву LogiHub, тобто було отримано лише 1 унікальну відповідь із 5.

При temperature=0.7 було отримано 4 унікальні відповіді з 5. Серед результатів були LogiChain, LogiHub, Logistika24 та LogiLink.

При temperature=1.5 усі п'ять відповідей відрізнялися, тобто було отримано 5 унікальних відповідей із 5. Водночас деякі назви стали незвичайними або некоректними та містили змішування символів різних мов.



```
C:\WINDOWS\system32\cmd.exe
(venv) C:\Users\oleks\lab1>python sampling.py
=== 1. ВПЛИВ TEMPERATURE НА ТВОРЧЕ ЗАВДАННЯ ===

temperature=0.0: унікальних 1 з 5
'LogiHub'
'LogiHub'
'LogiHub'
'LogiHub'
'LogiHub'

temperature=0.7: унікальних 4 з 5
'LogiChain'
'LogiHub'
'Logistika24'
'LogiLink'
'LogiHub'

temperature=1.5: унікальних 5 з 5
'DeaL_на лінія'
'LogiCart.'
'LogistiCloud'
'DeliveryMaximum'
'Бест@rіка'

=== 2. ВПЛИВ TOP_P НА ТВОРЧЕ ЗАВДАННЯ ===

top_p=0.1: унікальних 1 з 5
'LogiHub'
'LogiHub'
'LogiHub'
'LogiHub'
'LogiHub'

top_p=0.5: унікальних 2 з 5
'LogiFlow'
'LogiHub'
'LogiHub'
'LogiHub'
'LogiHub'

top_p=1.0: унікальних 3 з 5
'LogiHub'
```

Рисунок 25 – Вплив параметра temperature на різноманітність відповідей

Отриманий результат демонструє, що зі збільшенням temperature різноманітність відповідей моделі суттєво зросла: від 1 унікальної відповіді при temperature=0.0 до 5 при temperature=1.5.

2. Дослідження впливу top_p

Далі температура була зафіксована на рівні 0.7, а параметр top_p приймав значення 0.1, 0.5 та 1.0.

При top_p=0.1 було отримано 1 унікальну відповідь із 5. При top_p=0.5 кількість унікальних відповідей збільшилася до 2, а при top_p=1.0 — до 3.

Отже, збільшення top_p також підвищило різноманітність результатів, але його механізм відрізняється від temperature: top_p визначає набір кандидатів, доступних для вибору, тоді як temperature змінює співвідношення їхніх імовірностей.

3. Фактичне питання

Для фактичного питання було виконано по п'ять запусків при temperature=0.0 та temperature=1.5.

При temperature=0.0 модель п'ять разів відповіла:

1921
1921
1921
1921
1921

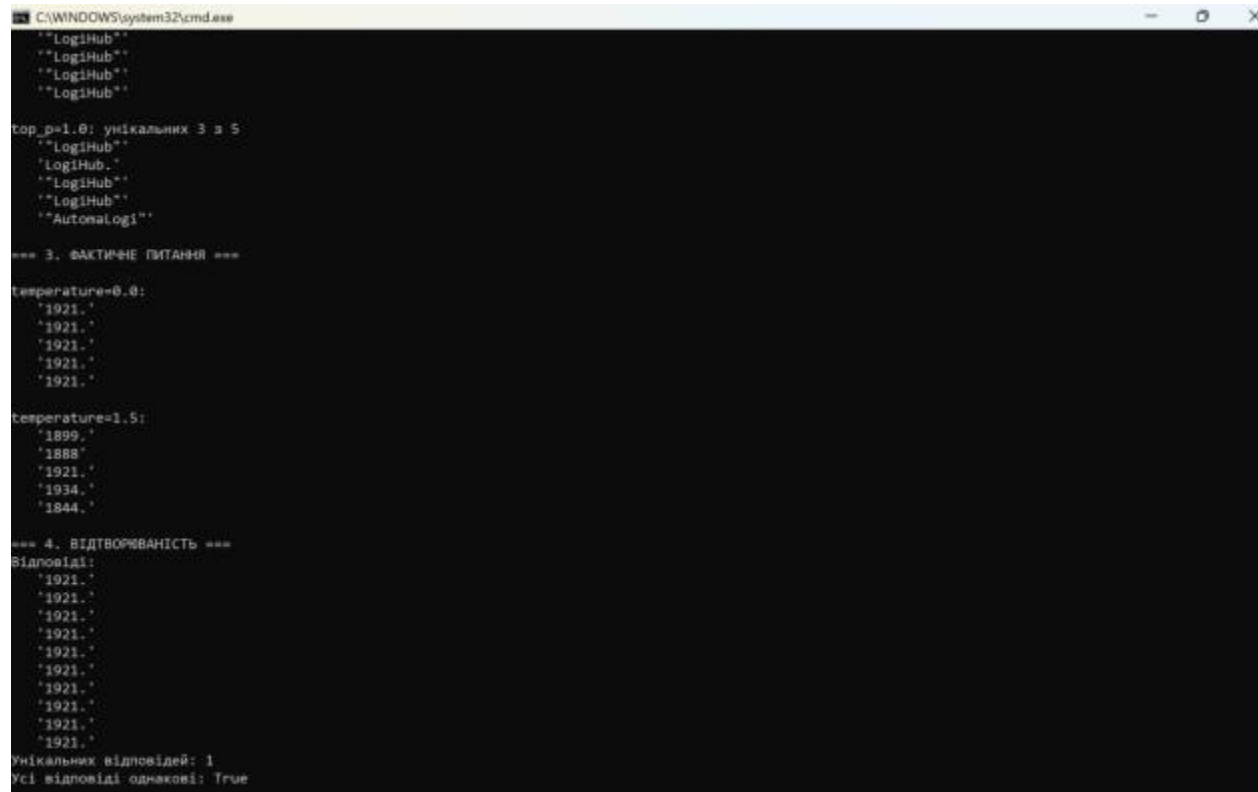
Однак правильний рік — 1918, тому кількість правильних відповідей становить 0 із 5. Odesa Polytechnic University

При temperature=1.5 було отримано:

1899
1888
1921
1934
1844

Жодна відповідь також не збігається з правильним роком 1918, тому результат становить 0 правильних із 5.

Цей експеримент показує важливу особливість: низька температура робить відповідь стабільнішою, але не робить неправильне знання правильним. При високій температурі модель почала генерувати різні неправильні роки.



```
C:\WINDOWS\system32\cmd.exe
""LogiHub""
""LogiHub""
""LogiHub""
""LogiHub""

top_p=1.0: унікальних 3 з 5
""LogiHub""
""LogiHub""
""LogiHub""
""LogiHub""
""AutomaLogi""

=== 3. ФАКТИЧНЕ ПИТАННЯ ===

temperature=0.0:
""1921.""
""1921.""
""1921.""
""1921.""
""1921.""

temperature=1.5:
""1899.""
""1888.""
""1921.""
""1934.""
""1844.""

=== 4. ВІДТВОРЮВАНІСТЬ ===
Відповіді:
""1921.""
""1921.""
""1921.""
""1921.""
""1921.""
""1921.""
""1921.""
""1921.""
""1921.""
""1921.""

Унікальних відповідей: 1
Усі відповіді однакові: True
```

Рисунок 26 – Результати фактичного питання та перевірки відтворюваності

4. Перевірка відтворюваності

Для перевірки відтворюваності фактичне питання було виконано 10 разів при:

temperature = 0.0

У всіх десяти випадках модель повернула:

1921

Таким чином, було отримано 1 унікальну відповідь із 10, а перевірка:

Усі відповіді однакові: True

показала повну відтворюваність саме в проведеному експерименті.

При цьому відповідь 1921 є фактично неправильною. Отже, відтворюваність результату не означає його правильність.

Таблиця 16 – Вплив параметрів на різноманітність відповідей

Параметр	Значення	Унікальних із 5 (творче)	Правильних із 5 (фактичне)
temperature	0,0	1	0
temperature	0,7	4	—
temperature	1,5	5	0
top_p	0,1	1	—
top_p	0,5	2	—
top_p	1,0	3	—

Таблиця 17 – Перевірка відтворюваності

Показник	Значення
Кількість запусків	10
temperature	0,0
Унікальних відповідей	1
Усі відповіді однакові	Так

Крок 7. Відповіді на питання

1. Яке значення температури ви поставите агенту, який має повертати JSON, і чому?

Для агента, який повинен повертати JSON, доцільно використовувати низьку температуру, наприклад temperature=0. Це зменшує випадковість і робить

формат відповіді більш передбачуваним, хоча правильність JSON усе одно потрібно перевіряти програмно.

2. Яке значення температури ви поставите генератору варіантів заголовків?

Для генерації різних варіантів заголовків доцільна вища температура, наприклад 0.7–1.0. Проведений експеримент показав, що при 0.7 було отримано 4 унікальні відповіді з 5, тоді як при 0.0 — лише одну.

3. Чим вплив `top_p` відрізняється від впливу `temperature` за характером, а не за силою?

`temperature` змінює співвідношення ймовірностей токенів, роблячи розподіл більш або менш випадковим. `top_p` натомість обмежує множину можливих токенів, відсікаючи менш імовірну частину розподілу.

4. Чи означає `temperature=0` повну відтворюваність результату? Що показав експеримент із 10 запусків?

Абсолютної гарантії відтворюваності `temperature=0` не дає, оскільки на результат можуть впливати особливості реалізації та обчислень. У проведеному експерименті всі 10 запусків дали однакову відповідь 1921, тобто було отримано одну унікальну відповідь.

5. Наведіть найбезглуздішу відповідь, яку модель видала при `temperature=1.5`, і поясніть, чому вона могла з'явитися.

Одним із найбільш незвичайних результатів була назва Вест 済 ріка, у якій модель змішала символи різних систем письма. При високій температурі збільшується шанс вибору малоімовірних токенів, тому модель може генерувати нестандартні, але водночас безглузді або мовно некоректні комбінації.

Крок 8. Питання до чат-бота

1. Що саме математично робить параметр `temperature` із розподілом ймовірностей наступного токена?

`Temperature` масштабує логіти моделі перед застосуванням функції `softmax`. Менше значення робить розподіл більш концентрованим навколо

найімовірніших токенів, а більше — вирівнює його та підвищує шанси менш імовірних варіантів.

2. Чим `top_p` відрізняється від `temperature` — обидва впливають на різноманітність, але за яким механізмом?

`temperature` змінює сам розподіл імовірностей між токенами. `top_p` формує обмежений набір найімовірніших токенів, сумарна ймовірність яких досягає заданого порогу, після чого вибір здійснюється лише серед них.

3. Чи є сенс змінювати обидва параметри одночасно, чи краще налаштовувати щось одне?

Технічно можна використовувати обидва параметри одночасно, але це ускладнює визначення причини зміни поведінки моделі. Для експериментального налаштування зручніше спочатку змінювати один параметр, залишаючи інший фіксованим.

4. Чому навіть при `temperature=0` мовна модель не гарантує абсолютно однакову відповідь при повторних запусках?

Навіть при мінімальній випадковості на результат можуть впливати особливості реалізації інференсу, паралельних обчислень і операцій із числами з плаваючою комою. У нашому експерименті ця потенційна нестабільність не проявилася — усі 10 відповідей були однаковими.

5. Які значення `temperature` використовують на практиці для витягу структурованих даних, а які для генерації творчого тексту?

Для структурованих і детермінованих завдань зазвичай використовують низьку температуру, близьку до 0. Для творчої генерації використовують вищі значення, наприклад близько 0.7–1.0, але конкретне оптимальне значення залежить від моделі та поставленого завдання.

Власні думки

Проведений експеримент показав, що для наступних лабораторних, у яких модель повинна повертати структуровані дані, доцільно використовувати низьку температуру. Вона зменшує випадковість і робить поведінку моделі більш стабільною, що особливо важливо при автоматичній обробці JSON та інших структурованих форматів.

Водночас низька температура не гарантує фактичної правильності. Це добре видно на фактичному питанні: при temperature=0 модель десять разів стабільно відповідала 1921, хоча офіційне джерело вказує 1918 рік. Odesa Polytechnic University

Налаштування за замовчуванням може бути непридатним, оскільки різні задачі мають різні вимоги. Для структурованих даних важлива передбачуваність, тоді як для генерації назв або ідей, навпаки, певна випадковість є корисною.

Висновок

У ході виконання Кроку 7 було експериментально досліджено вплив параметрів temperature і top_p на поведінку локальної мовної моделі. Зі збільшенням temperature кількість унікальних творчих відповідей зростає з 1 із 5 при temperature=0.0 до 5 із 5 при temperature=1.5. Для top_p також спостерігалось зростання різноманітності: від 1 із 5 при top_p=0.1 до 3 із 5 при top_p=1.0.

На фактичному питанні жоден із п'яти запусків при temperature=0.0 та temperature=1.5 не дав правильного року 1918. При високій температурі модель почала генерувати різні неправильні роки, а при нульовій — стабільно повторювала неправильний рік 1921.

Таким чином, temperature і top_p керують випадковістю та різноманітністю генерації, але не є регуляторами правильності моделі. Для наступних лабораторних зі структурованими даними доцільно використовувати низьку температуру та обов'язково програмно перевіряти отриманий результат.

Крок 8. Звести все разом і зробити висновок

На завершальному етапі було зведено результати всіх проведених експериментів. Порівнювалися локальна модель llama3.2:3b, локальна модель qwen3:4b та хмарна модель openai/gpt-oss-120b.

Таблиця 18 – Підсумкове порівняння моделей

Критерій	Локальна мала (llama3.2:3b)	Локальна більша (qwen3:4b)	Хмарна (openai/gpt-oss-120b)
Швидкість генерації	Середня; у тесті швидкості 11,4–39,9 символів/с, для довгої відповіді 32,6 символів/с	У цьому benchmark окремо не вимірювалася	Найвища; 65,9–1083,8 символів/с
TTFT	Близько 2,14–2,17 с після прогріву	У цьому benchmark окремо не вимірювався	Близько 0,31–0,44 с
Якість, балів /12	4/12	0/12	9/12
Стабільність формату (завдання 2)	Висока в проведеному тесті: обидві JSON-відповіді парсилися, хоча фактичні дані були неправильними	Низька: у початковому прогоні відповіді були порожні	Висока: обидві відповіді були валідним JSON
Потрібні ресурси	Ресурси локального ПК; модель приблизно 2,0 ГБ	Більше локальних ресурсів; модель приблизно 2,5 ГБ	Основні обчислення виконуються на сервері провайдера; локальні вимоги невеликі
Вартість	Після завантаження немає оплати за кожний API-запит; використовуються ресурси власного ПК	Аналогічно — використовуються ресурси власного ПК	Залежить від умов і тарифів хмарного API

Критерій	Локальна мала (llama3.2:3b)	Локальна більша (qwen3:4b)	Хмарна (openai/gpt-oss-120b)
Куди йдуть ваші дані	Обробляються локально на комп'ютері	Обробляються локально на комп'ютері	Запит передається зовнішньому хмарному API
Працює без інтернету	Так, після завантаження моделі	Так, після завантаження моделі	Ні, для API потрібне мережеве з'єднання
Коли обирати саме її	Коли важливі локальна обробка, автономність, контроль над даними та відсутність залежності від API, а нижча якість допустима	Коли потрібна локальна модель і є достатньо ресурсів для більшої моделі; у нашій конфігурації через нестабільні результати OpenAI-сумісного інтерфейсу перед практичним використанням потрібне додаткове тестування	Коли пріоритетом є швидкість і якість відповідей та допустимі залежність від інтернету і передавання запитів зовнішньому провайдеру

Примітка. Для qwen3:4b швидкість і TTFT у benchmark.py окремо не вимірювалися, тому ці значення не слід вигадувати. Також її результат 0/12 у початковому тесті якості значною мірою пов'язаний із порожніми відповідями в проведеній конфігурації, тому він не доводить, що модель загалом має нульову якість.

Аналіз результатів

За результатами вимірювання швидкості хмарна модель openai/gpt-oss-120b мала значно менший TTFT та вищу швидкість генерації порівняно з

локальною llama3.2:3b. Наприклад, для довгого запиту швидкість становила 1083,8 символів/с для хмарної моделі проти 32,6 символів/с для локальної.

Під час оцінювання якості за початковим quality.json модель llama3.2:3b набрала 4/12, qwen3:4b — 0/12, а хмарна модель — 9/12. Хмарна модель краще впоралася з кодом, логічним завданням та українською мовою.

Водночас експерименти продемонстрували, що навіть велика модель може давати неправильні фактичні відповіді.

Локальні моделі мають іншу важливу перевагу: запити можуть оброблятися без передавання їх зовнішньому API, а після завантаження моделей робота можлива без доступу до інтернету. Тому вибір моделі залежить не лише від швидкості або отриманого бала, а й від вимог конкретної системи.

Правило вибору моделі

Локальну модель доцільно обирати у випадках, коли важливими є автономність, локальна обробка даних, робота без інтернету та відсутність залежності від зовнішнього API. Це може бути важливішим за максимальну якість, якщо задача достатньо проста або отримані результати додатково перевіряються програмно.

Хмарну модель доцільно обирати, коли пріоритетними є висока швидкість і якість відповіді, а наявність інтернет-з'єднання та передавання запиту зовнішньому сервісу допустимі. Для задач із чутливими або конфіденційними даними перед вибором хмарної моделі необхідно окремо враховувати правила обробки даних конкретного провайдера.

Загальний висновок до лабораторної роботи

У ході лабораторної роботи було розгорнуто середовище для роботи з локальними та хмарними мовними моделями. Було налаштовано локальний сервер Ollama та OpenAI-сумісний доступ до хмарної моделі, після чого проведено серію експериментів із моделями llama3.2:3b, qwen3:4b та openai/gpt-oss-120b.

Порівняння показало суттєві відмінності у швидкості. Для llama3.2:3b медіанний TTFT після прогріву становив приблизно 2,14–2,17 с, тоді як для хмарної моделі — приблизно 0,31–0,44 с. На довгому запиті швидкість генерації становила 32,6 символів/с для локальної моделі та 1083,8

символів/с для хмарної. Також було встановлено вплив холодного запуску: TTFT локальної моделі змінився з 8,159 с при холодному запуску до 2,285 с після прогріву.

Оцінювання за рубрикою продемонструвало, що швидкість сама по собі не характеризує якість мовної моделі. У початковому прогоні llama3.2:3b отримала 4/12, qwen3:4b — 0/12, а хмарна модель — 9/12. Особливо показовими виявилися завдання на фактичні знання, багатокрокове міркування та якість української мови. При цьому результати qwen3:4b слід трактувати обережно через нестабільність відповідей у використаній конфігурації.

Експеримент із temperature показав прямий вплив цього параметра на різноманітність генерації. При temperature=0.0 було отримано 1 унікальну творчу відповідь із 5, при 0.7 — 4 із 5, а при 1.5 — 5 із 5. Збільшення top_p також підвищило різноманітність: від 1 унікальної відповіді при top_p=0.1 до 3 при top_p=1.0.

Водночас збільшення випадковості не покращило фактичну точність. На питанні про рік заснування Одеської політехніки при temperature=0 модель стабільно відповідала 1921, а при temperature=1.5 генерувала різні роки. Це показало, що низька температура підвищує передбачуваність, але не гарантує правильності знань моделі.

Отже, універсально найкращої моделі або універсально найкращих параметрів генерації немає. Для наступних лабораторних зі структурованими даними доцільно використовувати низьку temperature, перевіряти формат програмно та обирати між локальною і хмарною моделлю відповідно до вимог конкретної задачі. Локальні моделі доцільні за потреби автономності та локальної обробки, тоді як хмарна модель у проведених експериментах забезпечила кращі показники швидкості та якості.